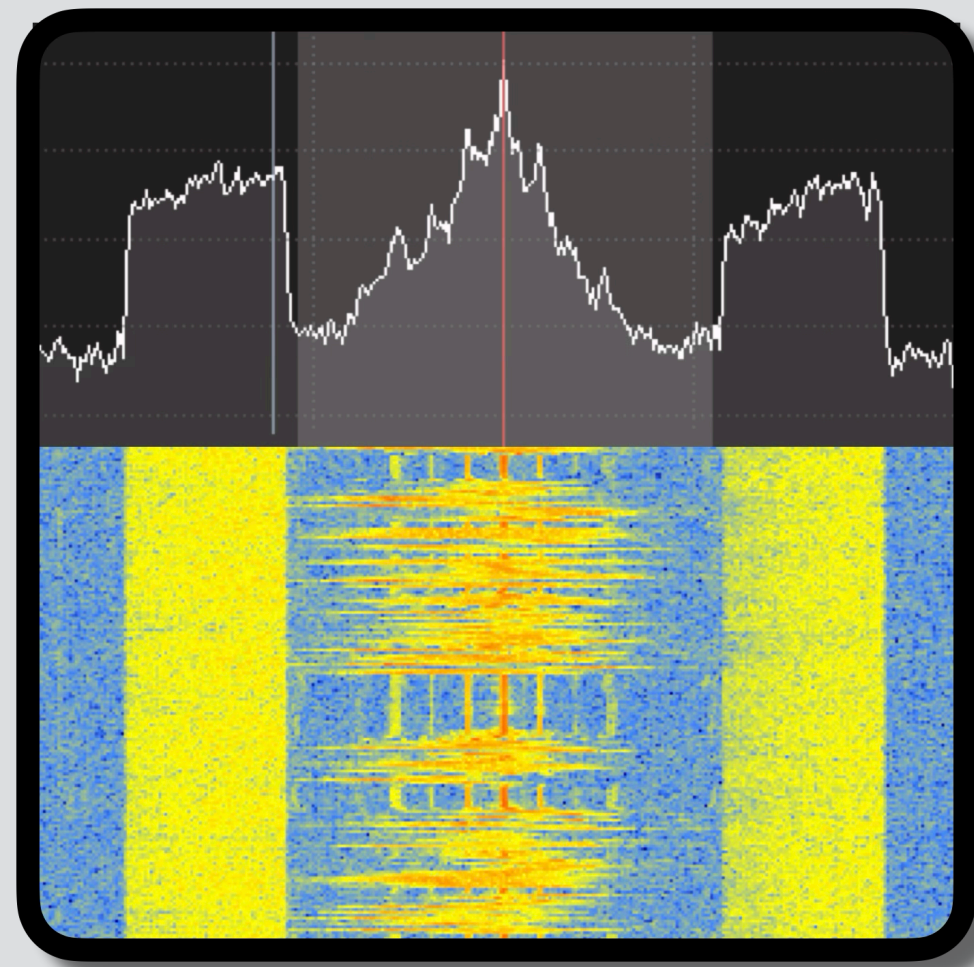
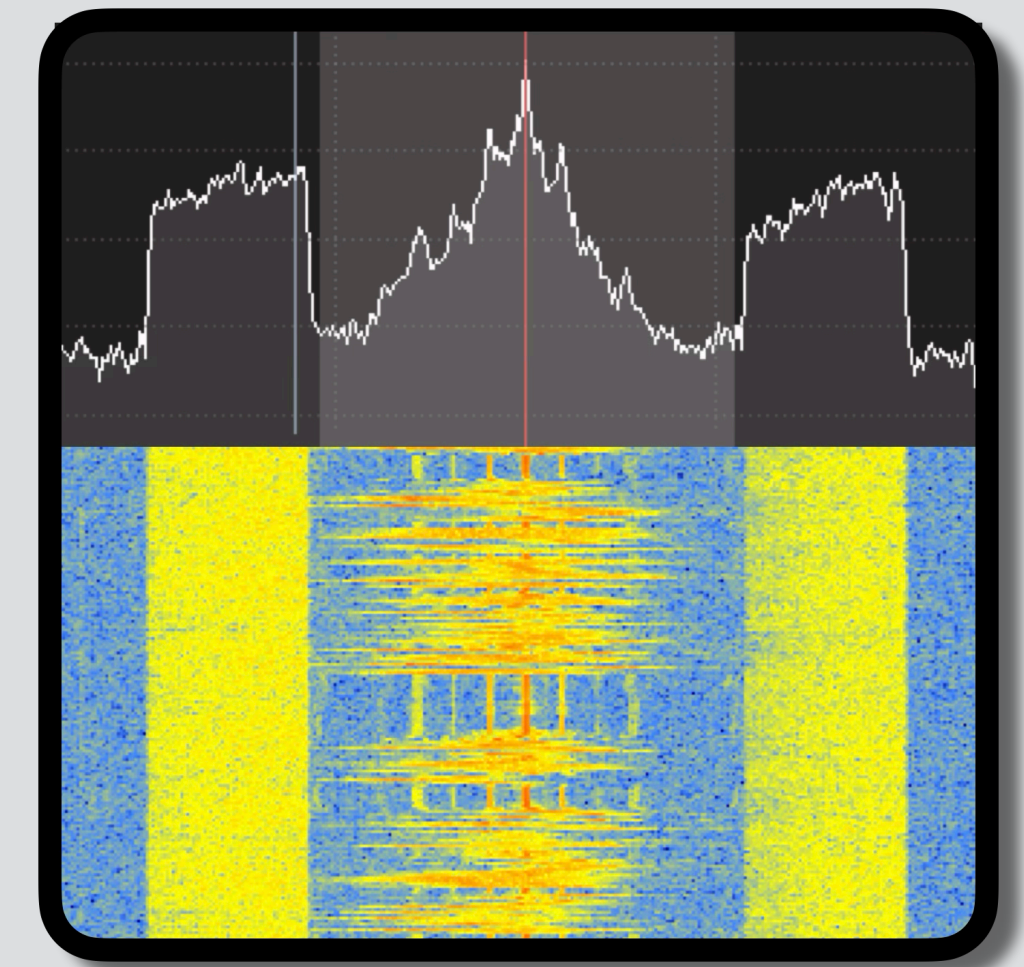




EE121

& Analog



Introduction to Digital Communication Systems

Lecture 12A

Miki Lustig

Electrical Engineering and Computer Sciences, UC Berkeley

Today

- Last time:
 - Error Detection and Correction
 - Parity and CRC
 - Coding
- Today: Andrews notes (Bcourses)
 - Convolutional Codes
 - Viterbi Decoding
 - Soft Decoding

Error Protection: Detection and Correction

- Communications channels are subject to noise
- Noise can cause digital signals to be received as different values
 - Bits can be flipped
 - Points in the constellation can shift
- Given errors:
 - We would like to detect or correct
 - Add redundancy to do that

Parity Check

- Used to verify the integrity of a block
- Append a check bit to the data so codeword have the same parity — even or odd

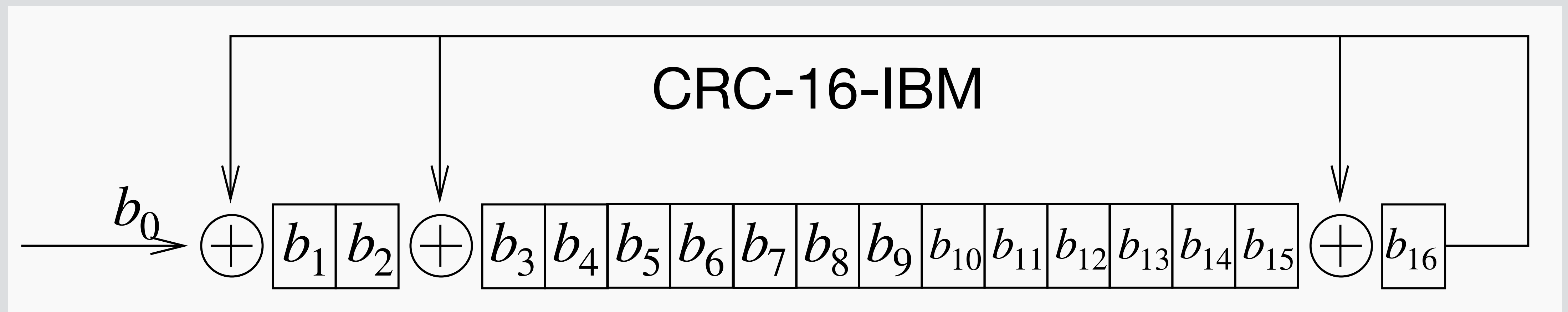
$$c_1 \oplus c_2 \oplus \cdots \oplus c_N \oplus c_p = 0 \quad \Rightarrow \quad c_1 \oplus c_2 \oplus \cdots \oplus c_N = c_p$$

$$c_1 \oplus c_2 \oplus \cdots \oplus c_N \oplus c_p = 1 \quad \Rightarrow \quad c_1 \oplus c_2 \oplus \cdots \oplus c_N \oplus 1 = c_p$$

- Any single bit (or odd number) error can be detected

Cyclic Redundancy Check (CRC)

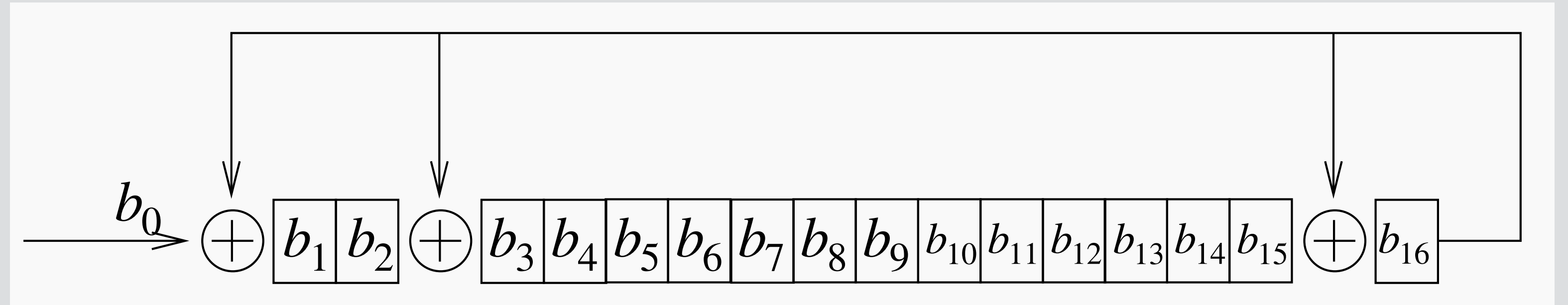
- CRC generators are implemented as a linear-feedback shift registers with XOR operations at various locations



$$g(D) = 1 + D^2 + D^{15} + D^{16}$$

- Polynomial: Every delay is power of 2 - some powers are XORed

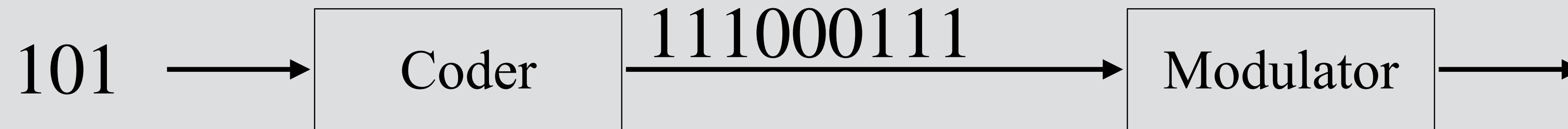
CRC Error Detection



- At TX:
 - Whole packet of bits is input, in serial to the generator
 - The state at the end of the packet is: $b = (b_1, b_2, \dots, b_{16})$
 - Transmit 16 CRC bits at the end of the packet as a designated part (link layer)
- At RX:
 - Decode the packet
 - Run the decoded bits through the $g(D)$, the CRC generator
 - Compare against CRC part of the packet.

Repetition Codes

Consider repetition code rate $r=1/3$, BPSK. SNR=3dB:



$$P_e = Q(\sqrt{2}) = 0.079 \triangleq p$$

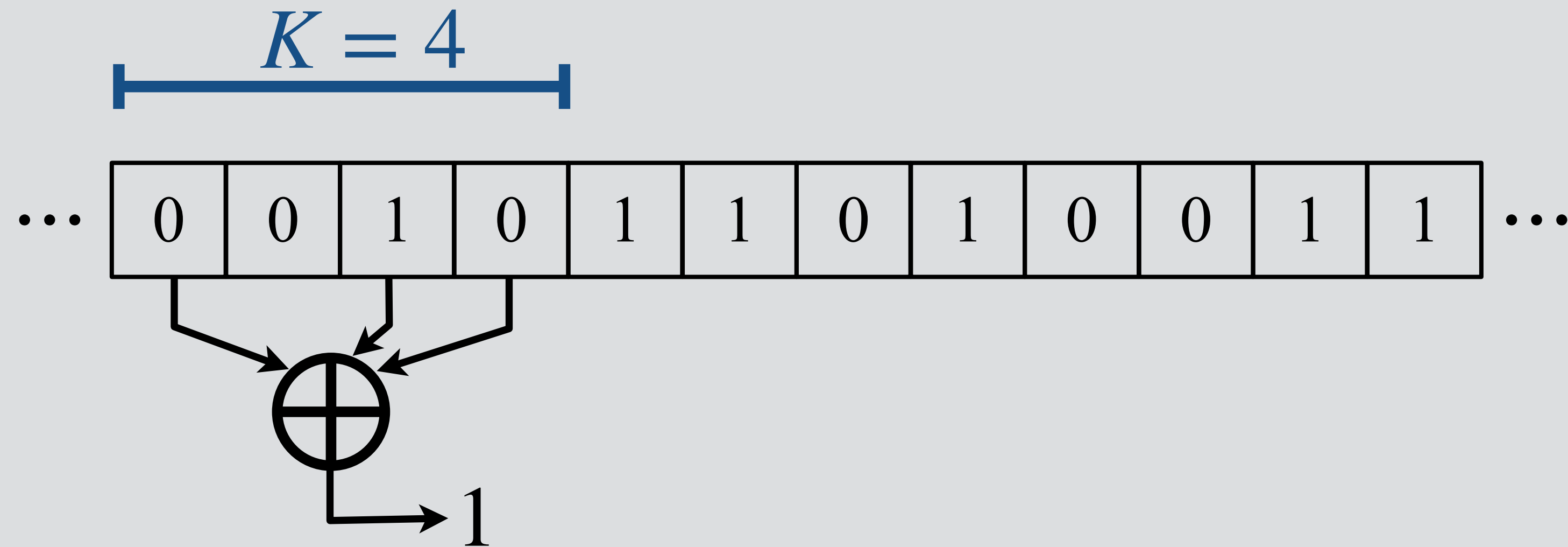
At RX, we detect each bit (i.i.d noise) and decode with majority vote

$$101 \rightarrow 1, \quad 001 \rightarrow 0$$

Convolutional Codes

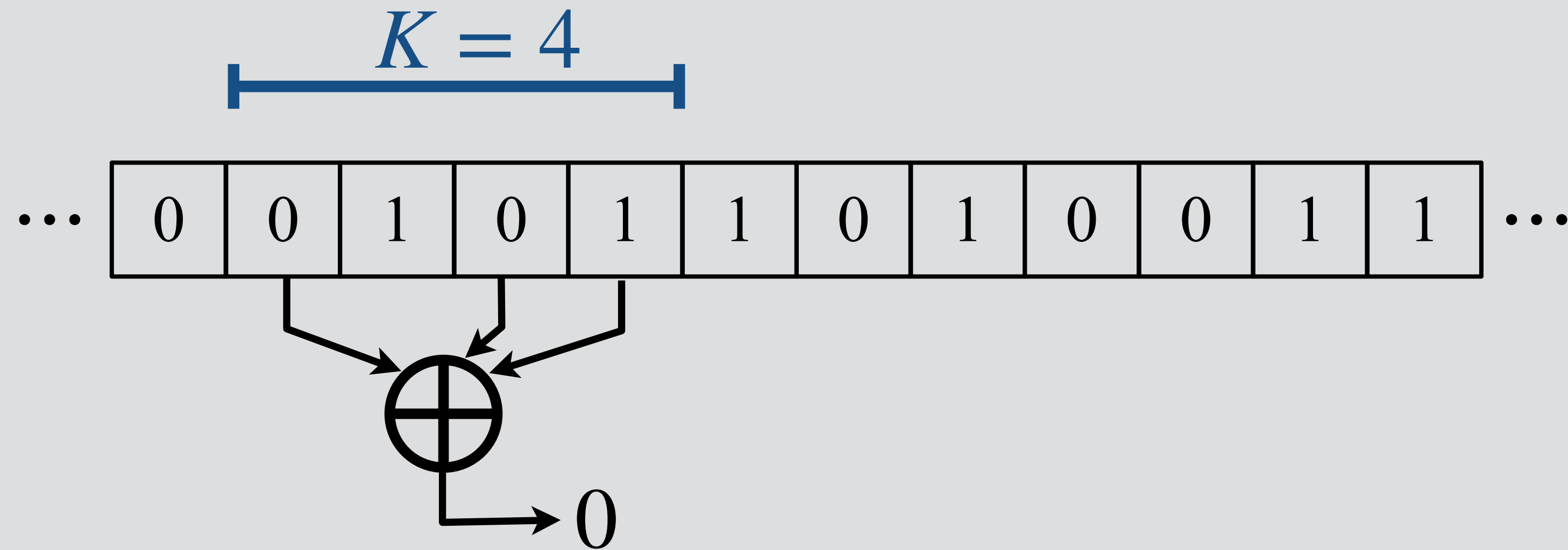
- A set of modulo two filters convolved with input stream
 - Add correlation between output bits
 - Correlation can be used to correct errors at the receiver
- A sliding parity bit calculation
 - Transmit only the parity bits

Convolutional Codes Mechanics



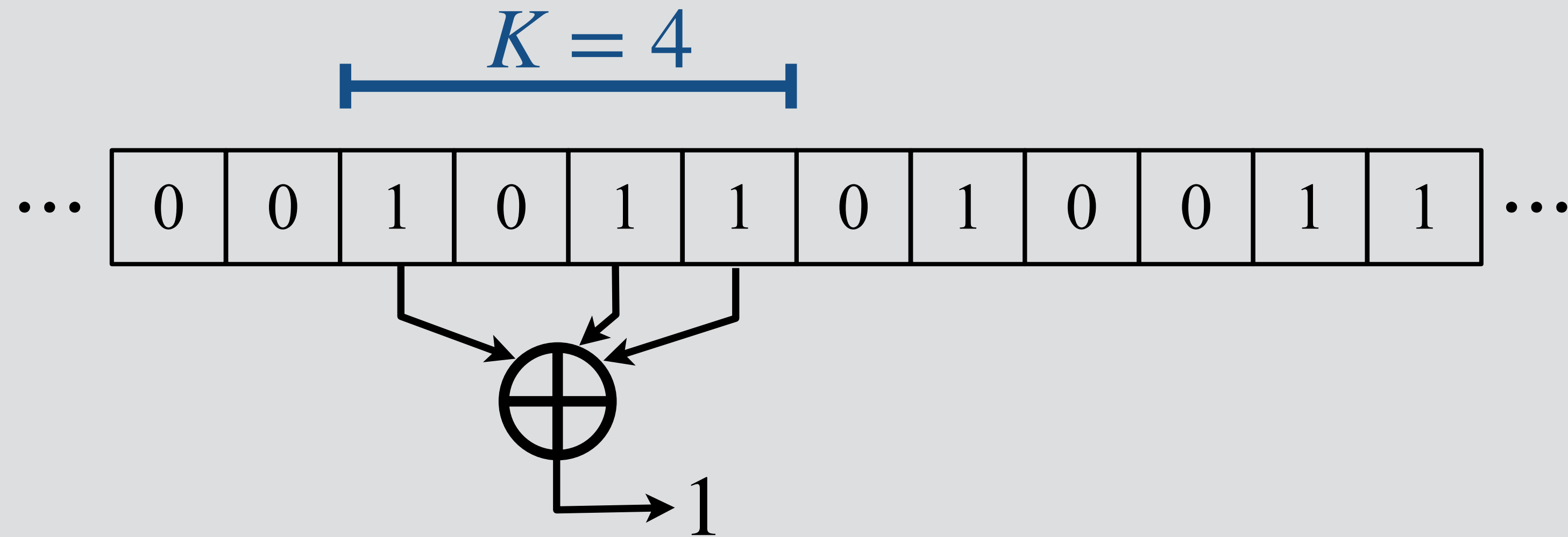
- Output: 1

Convolutional Codes Mechanics



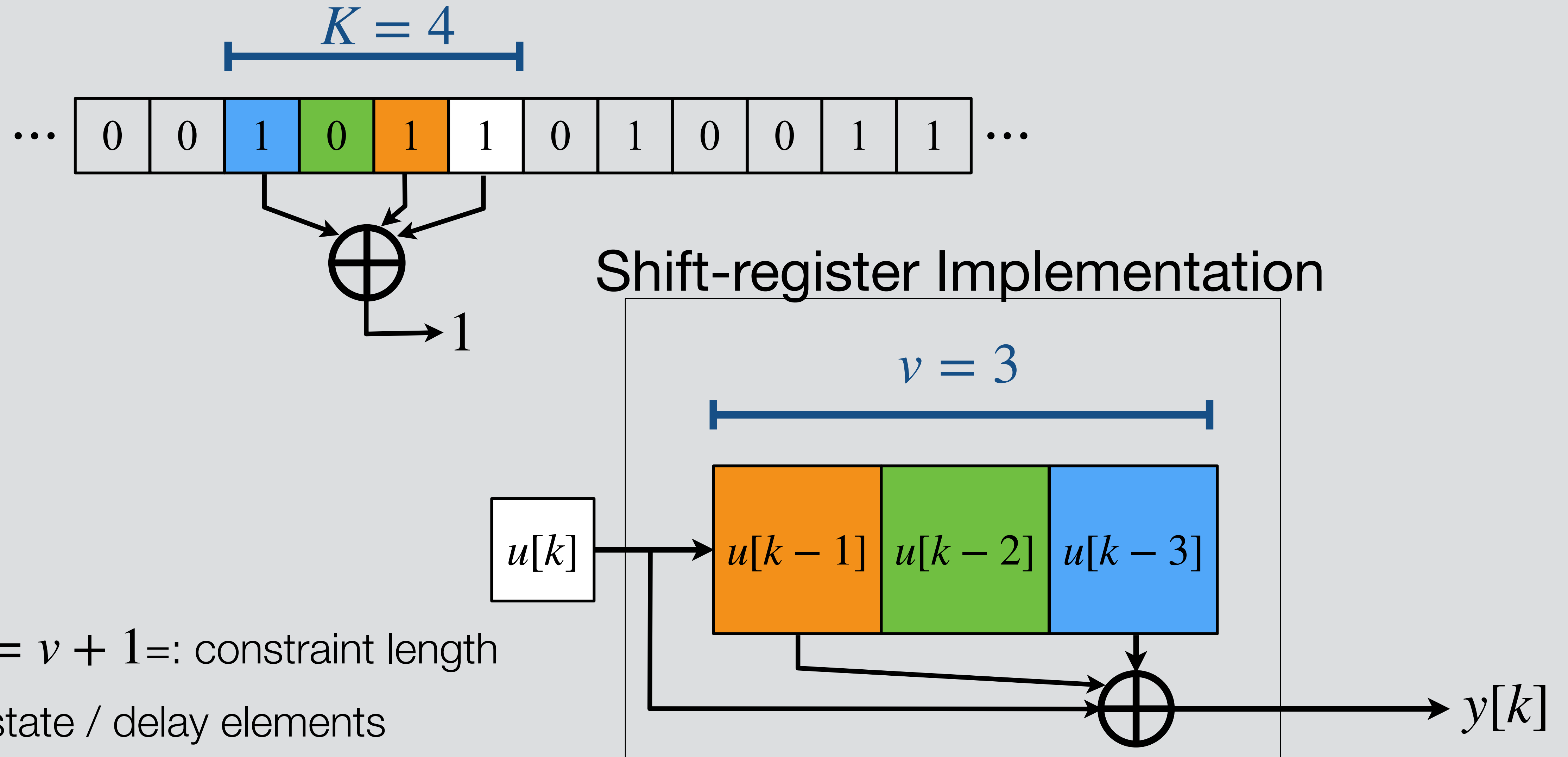
- Output: 1 0

Convolutional Codes Mechanics

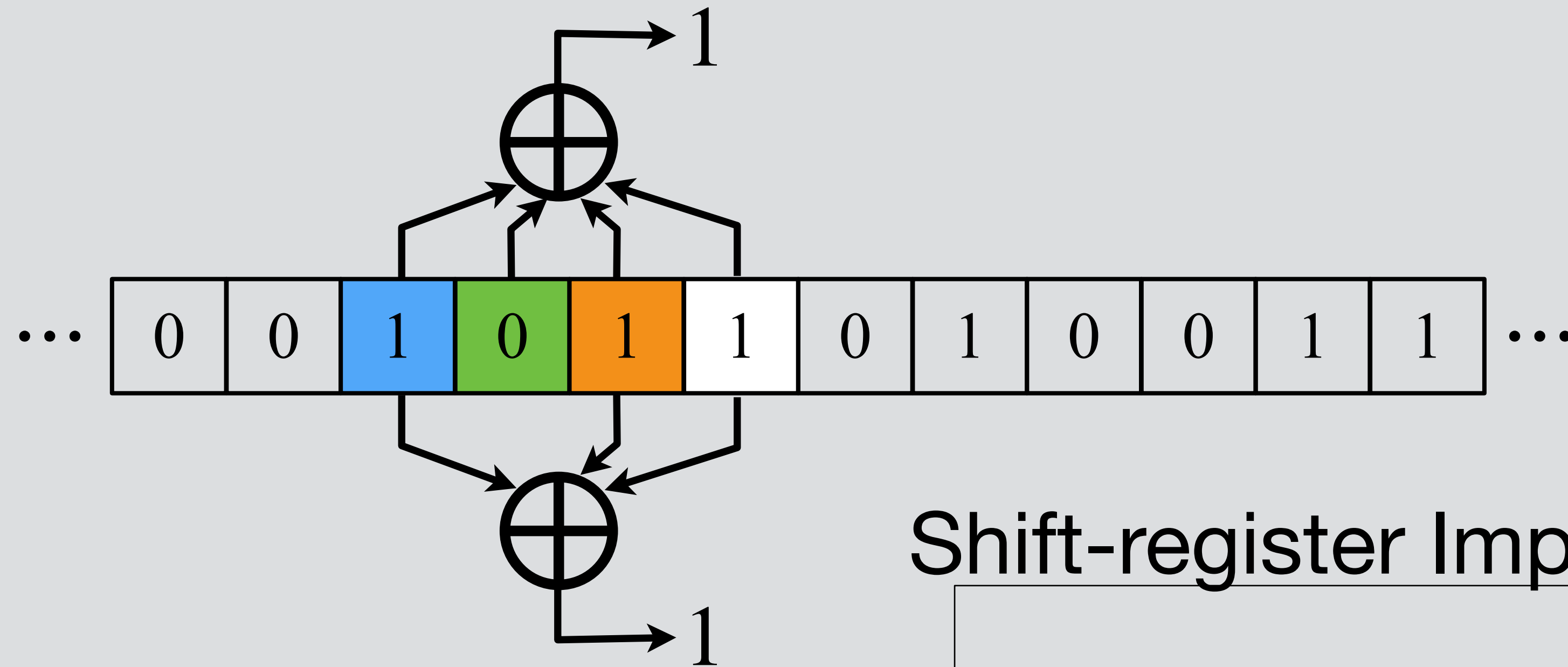


- Output: 1 0 1

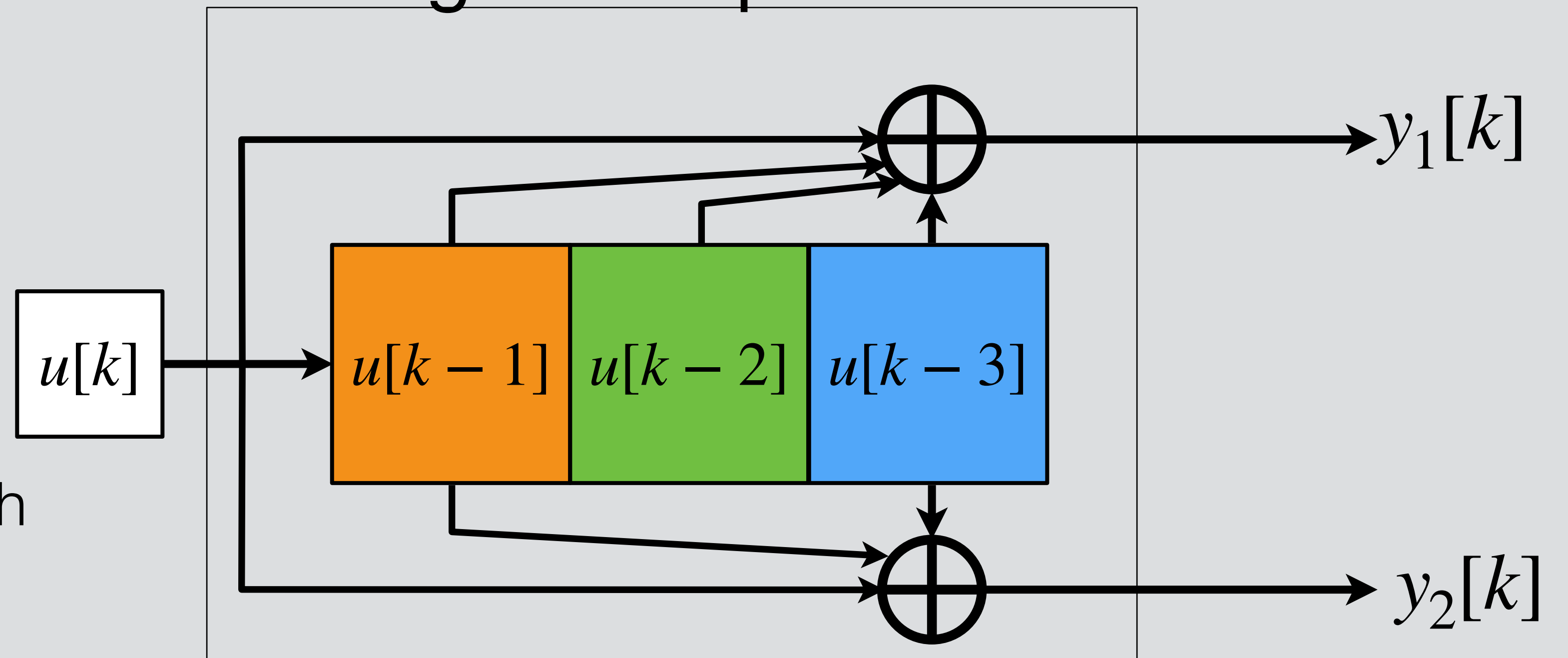
Convolutional Codes Mechanics



Convolutional Codes Mechanics



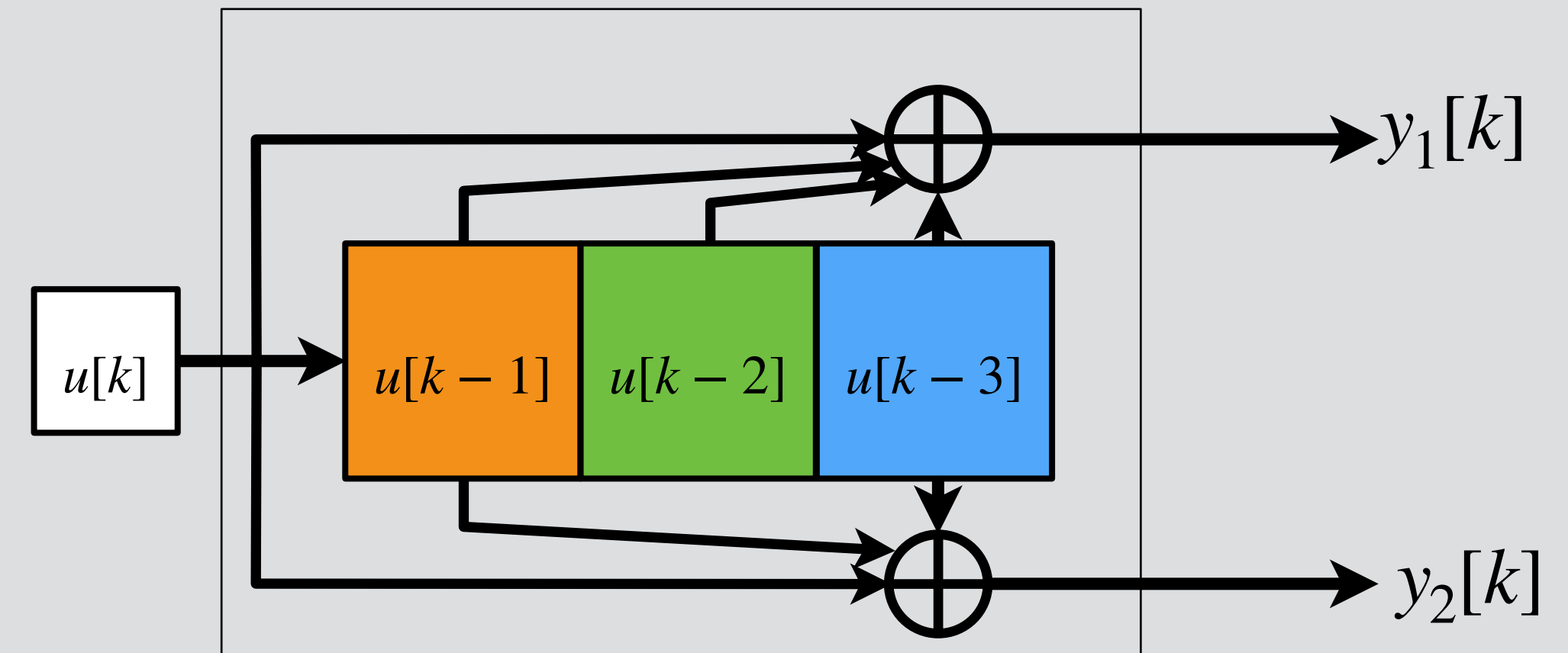
Shift-register Implementation



- $K = v + 1$ =: constraint length
- v : state / delay elements

Convolutional Codes

- Coding Generator matrix $G(D)$
 - $G(D)$ is $m \times n$ (here 4×2)
 - $G(D) = [g_{11}(D) \quad g_{12}(D)]$
- State: $s[k] = (u[k-1], u[k-2], u[k-3])$



$$g_{11}(D) = 1111_{(2)} = 17_{(8)} = 1 + D + D^2 + D^3$$

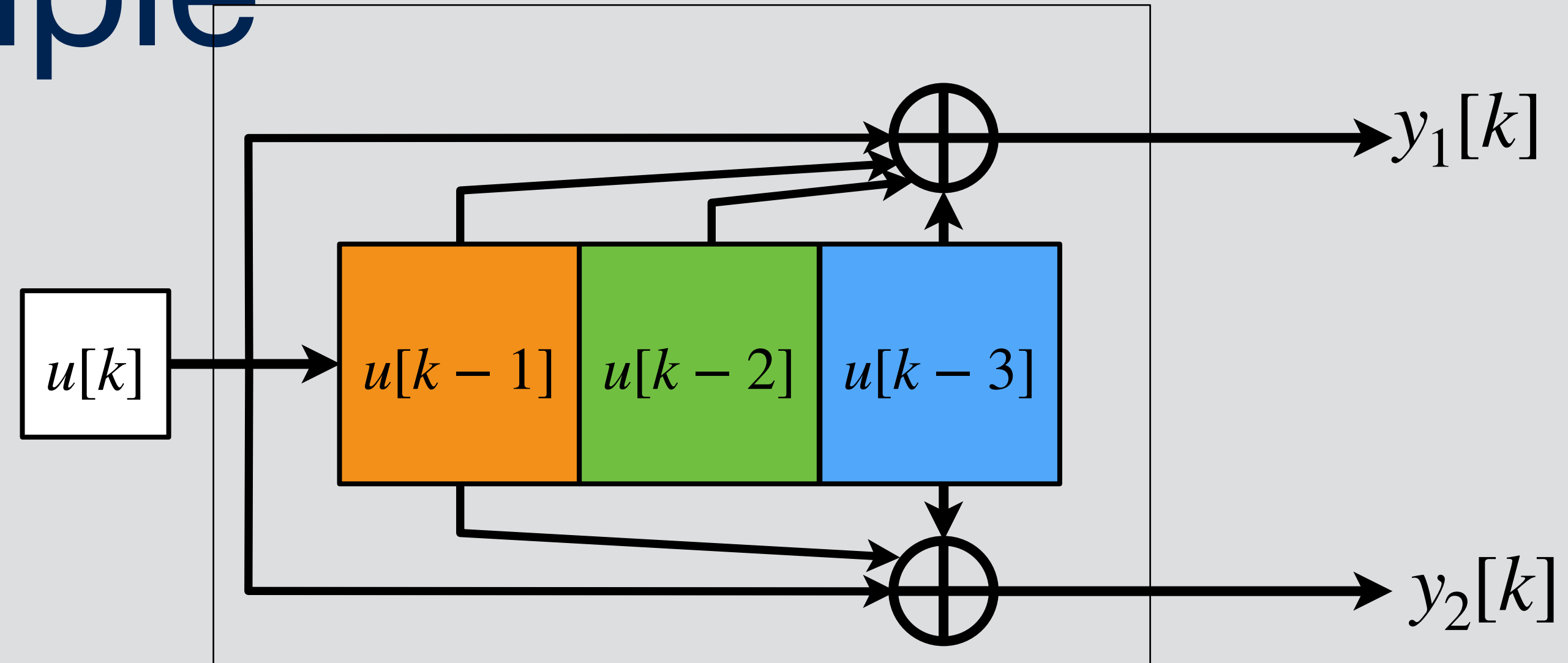
$$g_{12}(D) = 1101_{(2)} = 15_{(8)} = 1 + D + D^3$$

$$y_1[k] = u[k] \oplus u[k-1] \oplus u[k-2] \oplus u[k-3]$$

$$y_2[k] = u[k] \oplus u[k-1] \oplus u[k-3]$$

Example

k	$u[k]$	$s[k]$	$y_1[k]$	$y_2[k]$
0	1	(0,0,0)		
1	0			
2	1			
3	1			
4	0			
5	1			



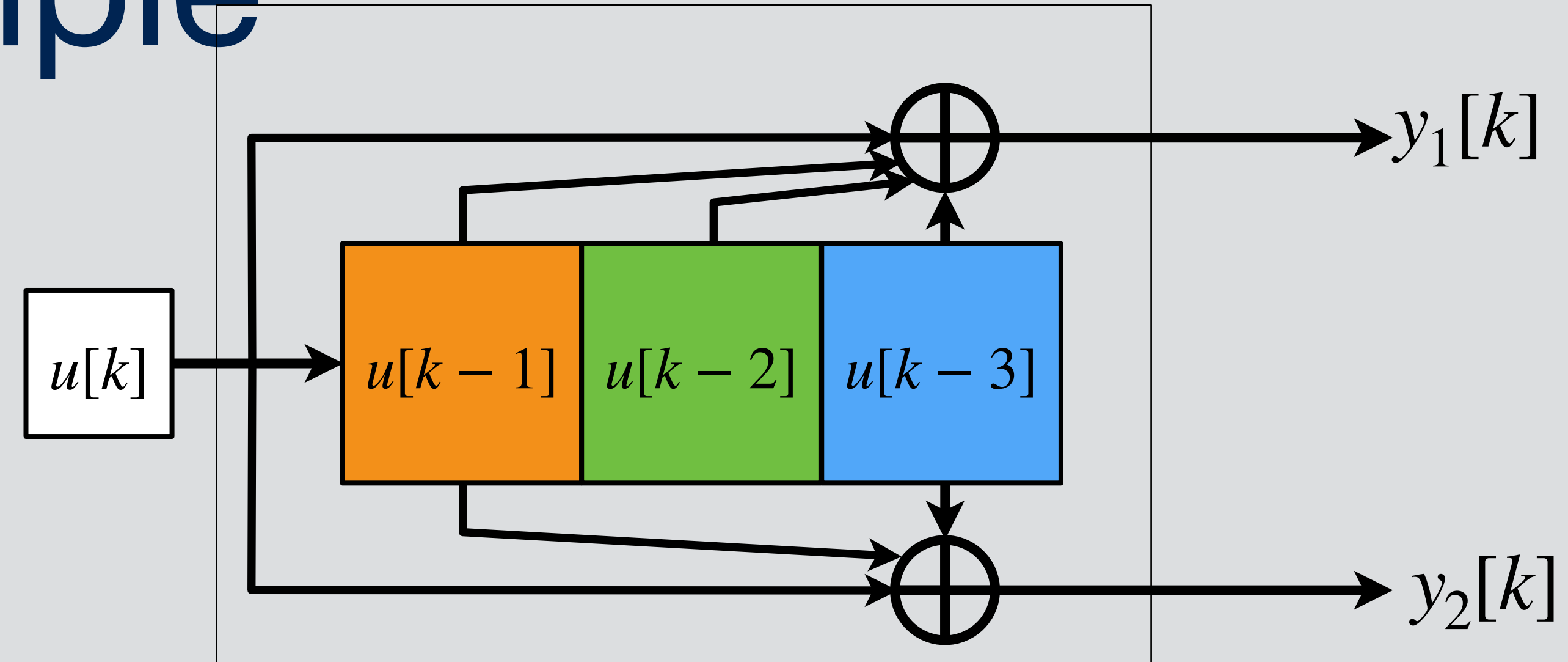
$$g_{11}(D) = 1111_{(2)}$$

$$g_{12}(D) = 1101_{(2)}$$

$$[u[k] \ s[k]] \begin{bmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix} = [y_1[k] \ y_2[k]]$$

Example

k	$u[k]$	$s[k]$	$y_1[k]$	$y_2[k]$
0	1	(0,0,0)	1	1
1	0	(1,0,0)	1	1
2	1	(0,1,0)	0	1
3	1	(1,0,1)	1	1
4	0	(1,1,0)	0	1
5	1	(0,1,1)	1	0

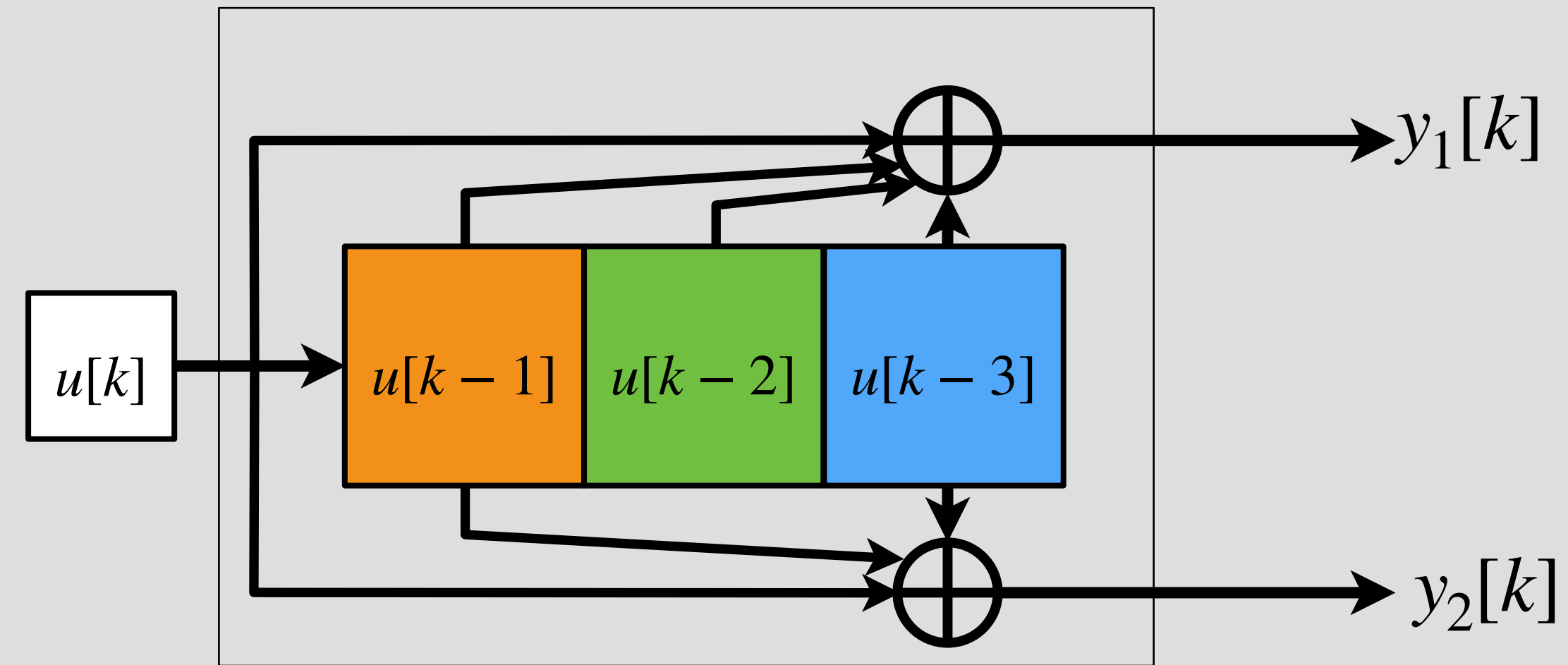
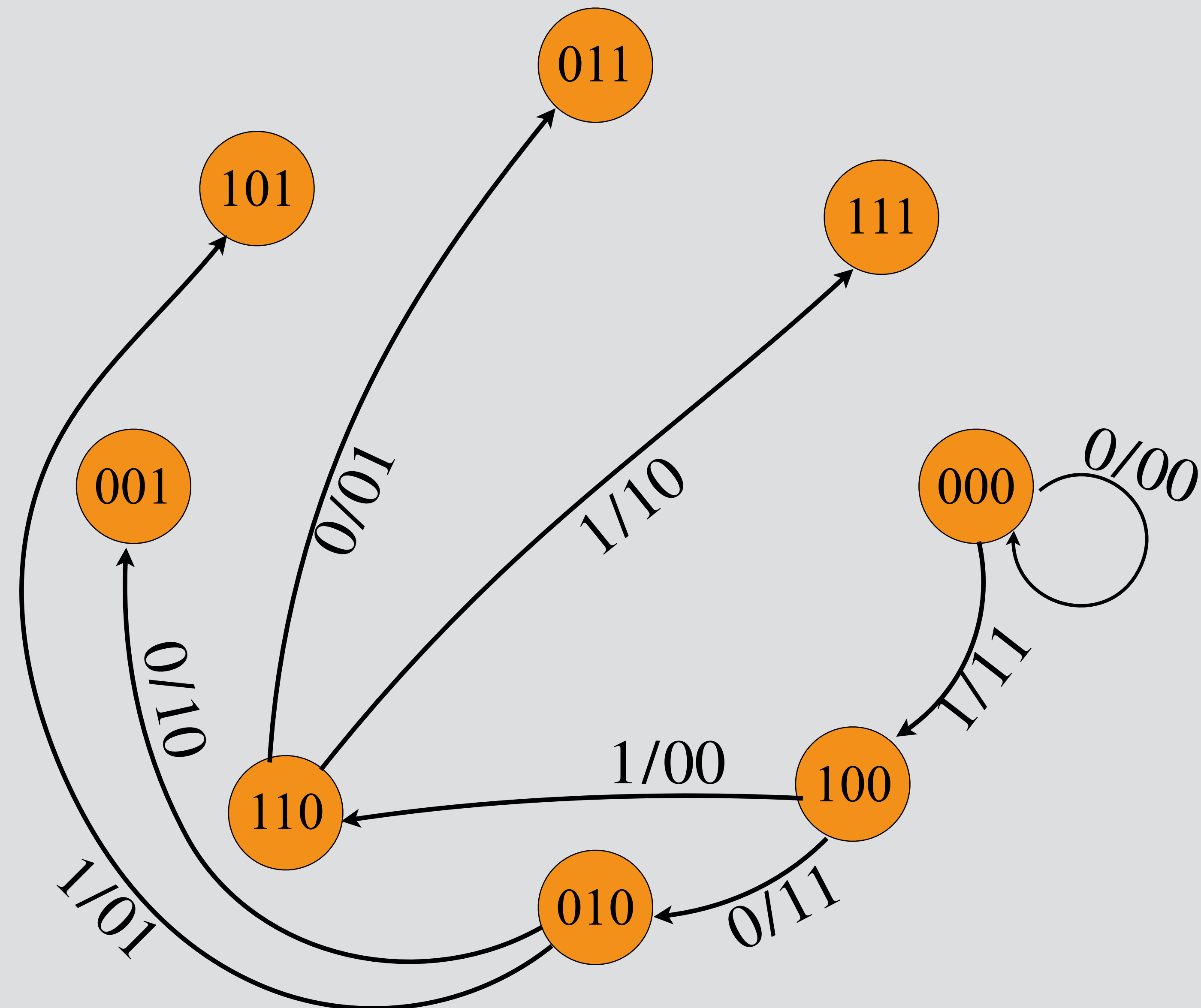


$$g_{11}(D) = 1111_{(2)}$$

$$g_{12}(D) = 1101_{(2)}$$

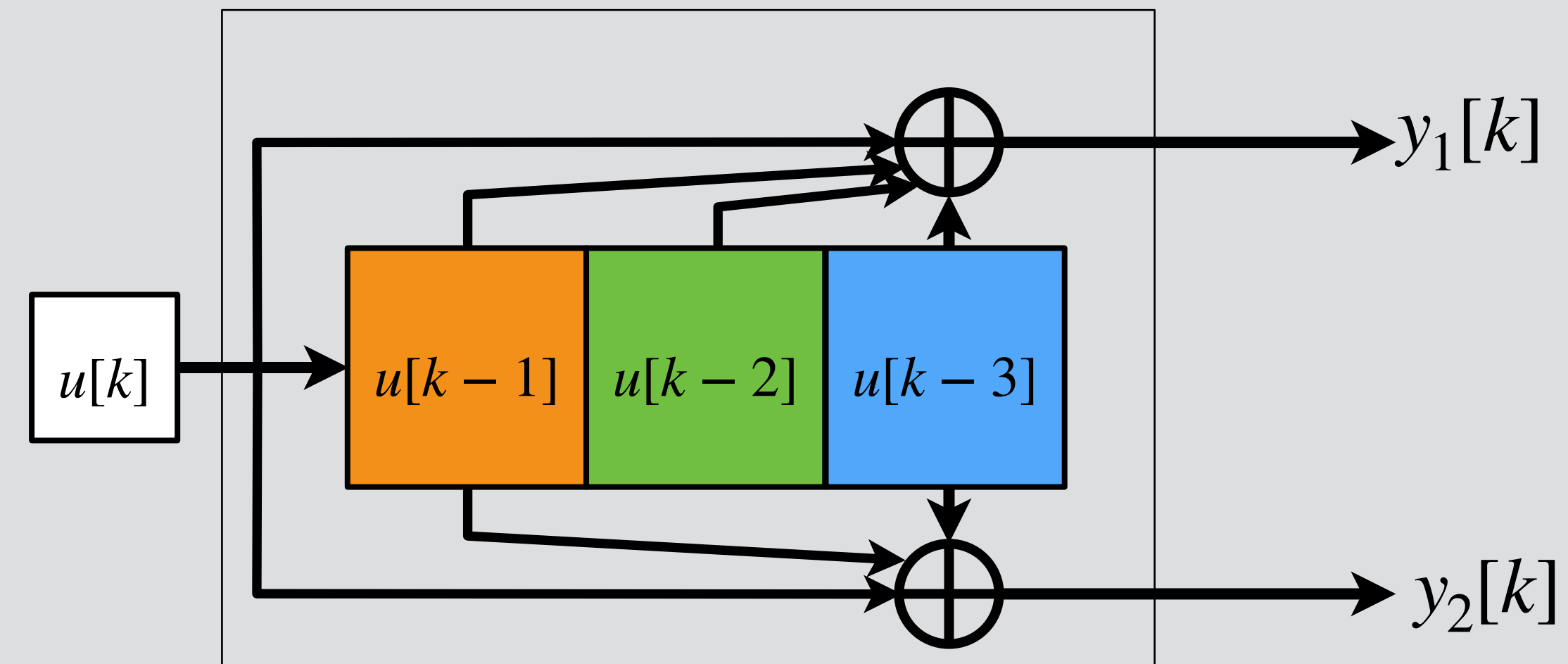
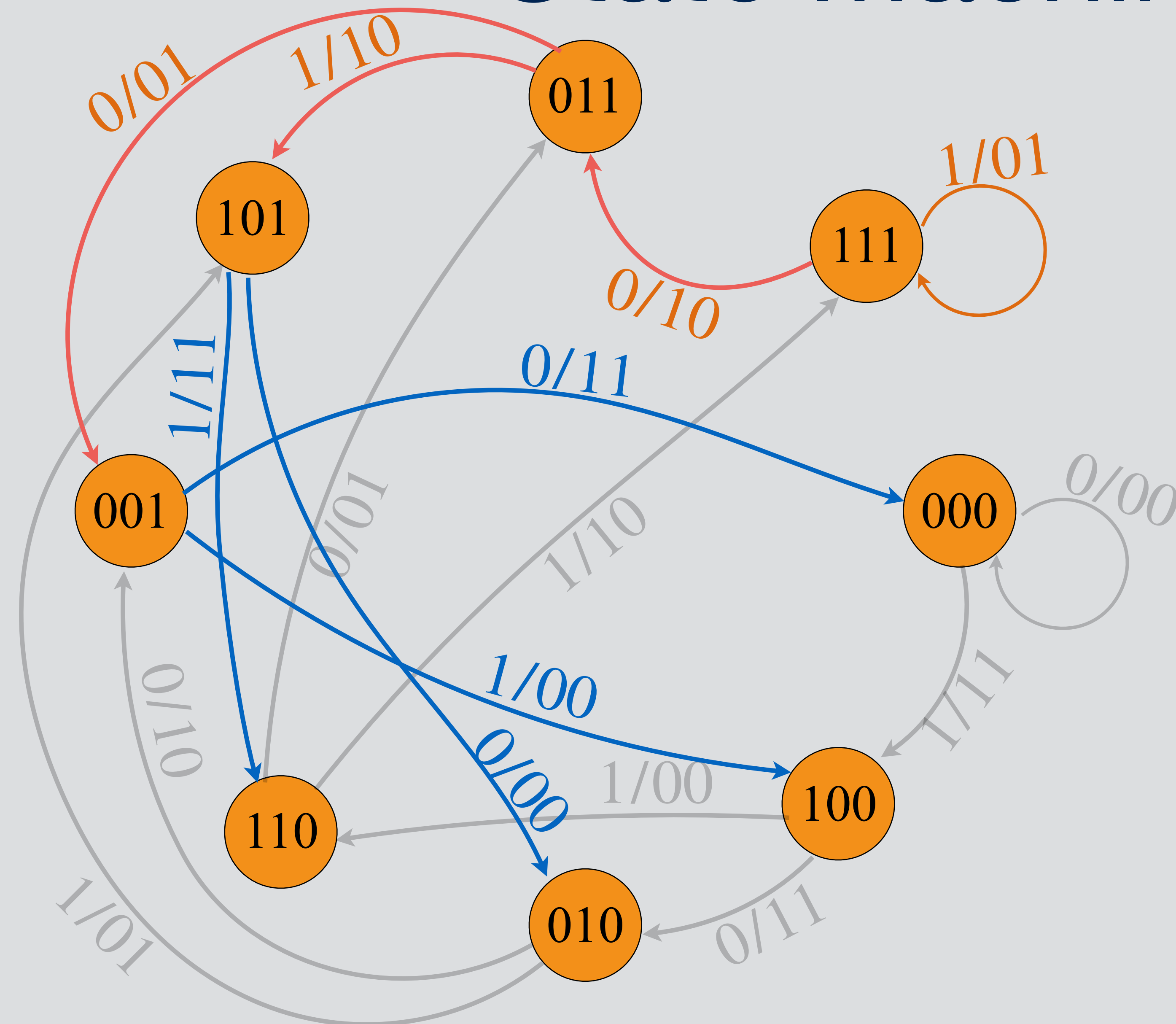
$$[u[k] \ s[k]] \begin{bmatrix} 1 & 1 \\ 1 & 1 \\ 1 & 0 \\ 1 & 1 \end{bmatrix} = [y_1[k] \ y_2[k]]$$

State Machine View



- $K = 4, r = 1/2$
- 8 states
- Arcs: u/y_1y_2

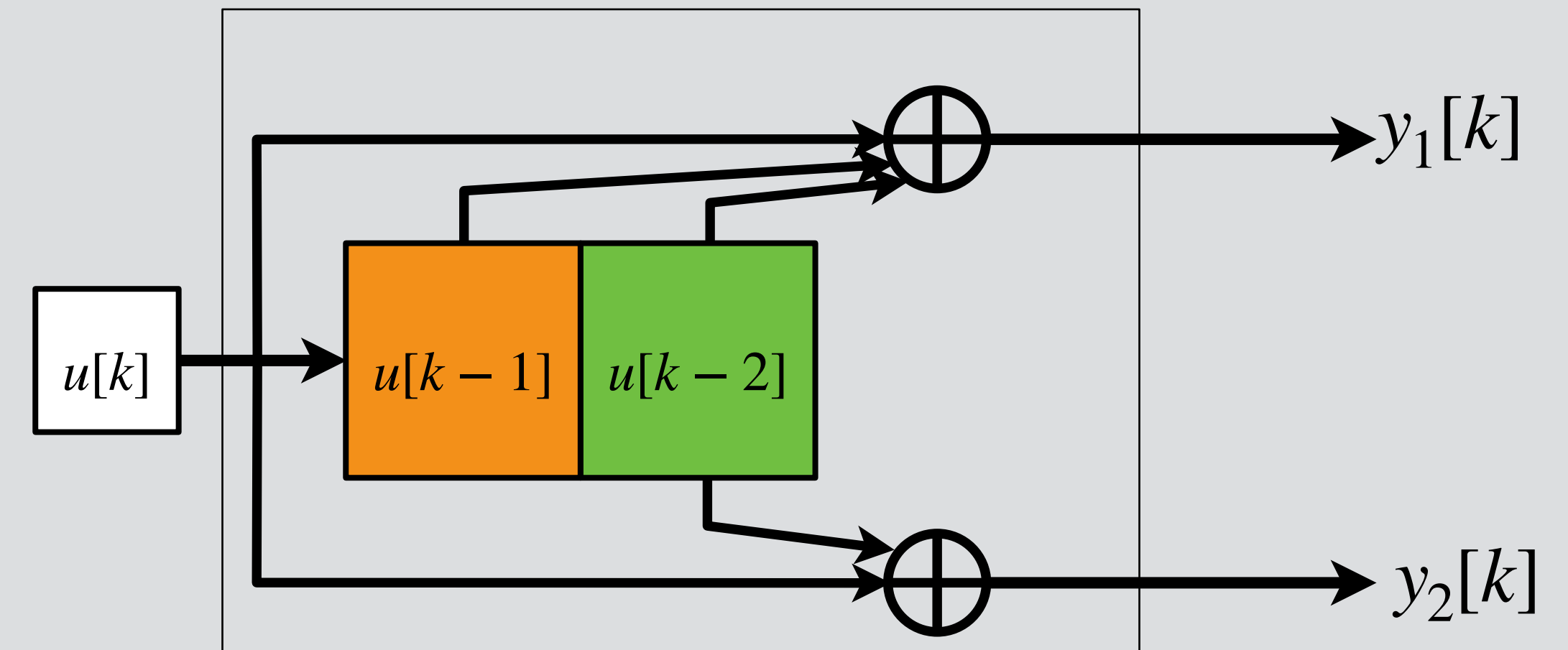
State Machine View



- $K = 4, r = 1/2$
- 8 states
- Arcs: u/y_1y_2

Towards Decoding

Message	Coded bits	Distance
“000”	“000000”	
“001”	“000011”	
“010”	“001110”	
“011”	“001101”	
“100”	“111011”	
“101”	“111000”	
“110”	“110101”	
“111”	“110110”	



received: $\hat{y} = 101110$

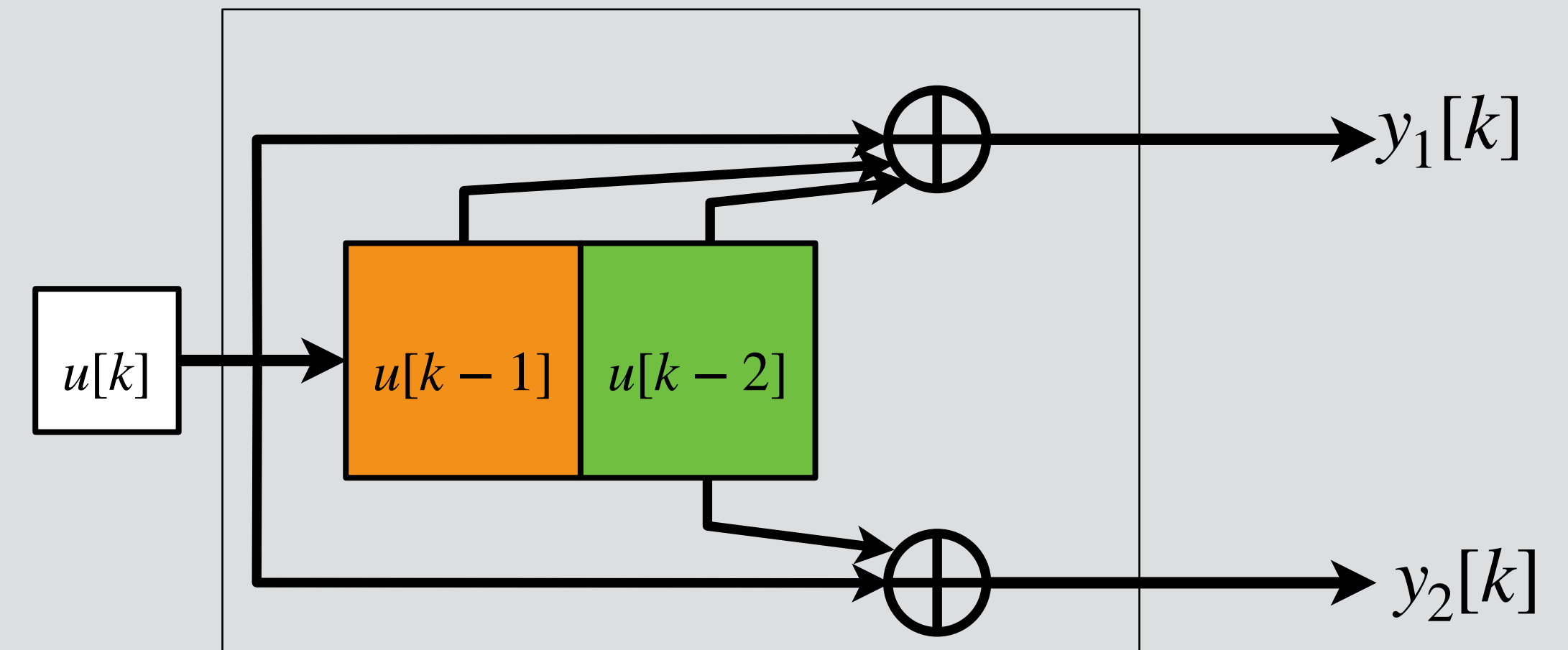
Q: Which 3 bits were transmitted?

A: Compute hamming distance:

$$d_{\text{hamming}} = \sum_{i=0}^5 \hat{y}[i] \oplus y[i]$$

Towards Decoding

Message	Coded bits	Distance
"000"	"000000"	4
"001"	"000011"	4
"010"	"001110"	1
"011"	"001101"	3
"100"	"111011"	3
"101"	"111000"	3
"110"	"110101"	4
"111"	"110110"	2



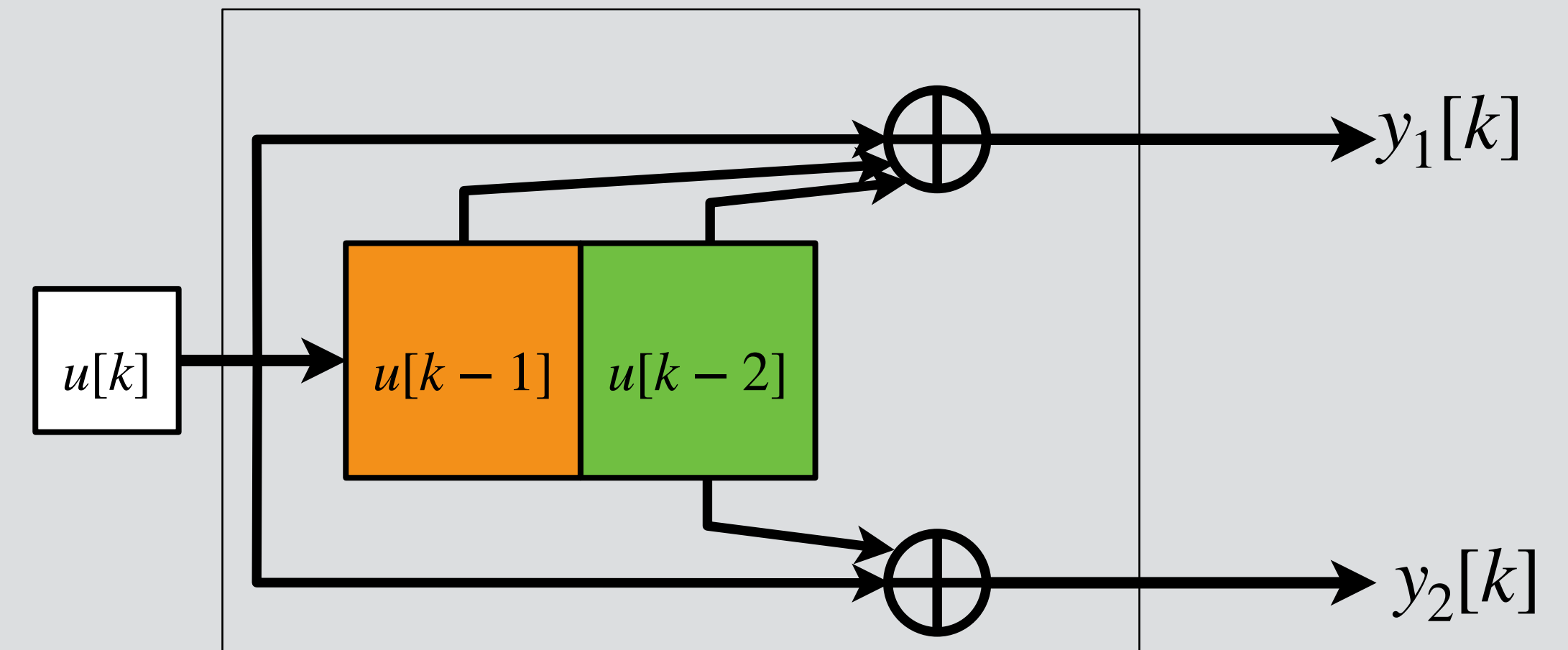
received: $\hat{y} = 101110$

Q: Which 3 bits were transmitted?

A: Compute hamming distance:

$$d_{\text{hamming}} = \sum_{i=0}^5 \hat{y}[i] \oplus y[i]$$

Towards Decoding

[illegible]

received: $\hat{y} = 1110101101$

Q: Which 5 bits were transmitted?

Need to compute 2^5 possible codewords

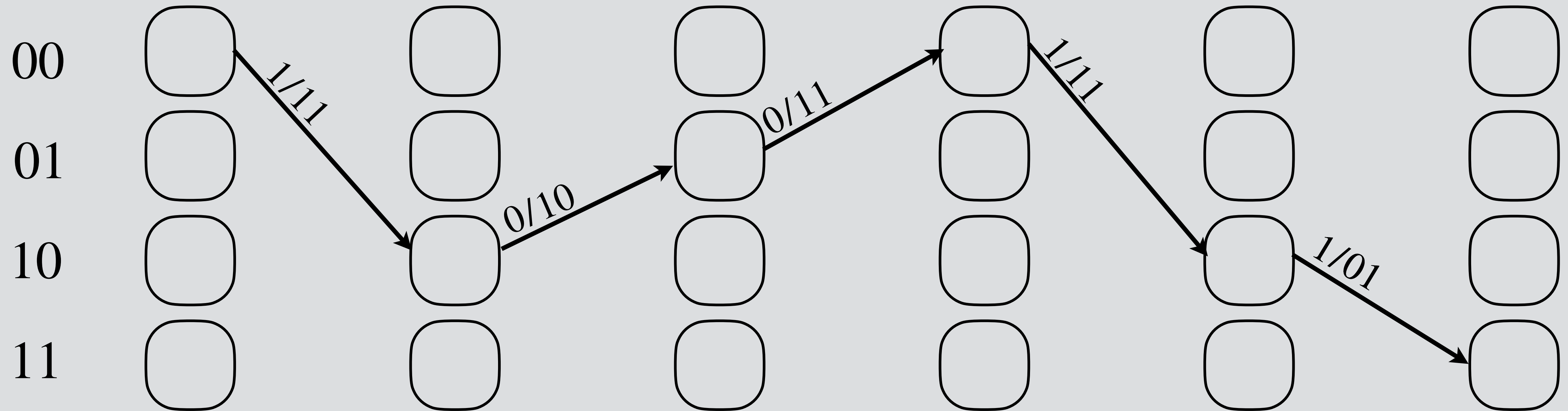
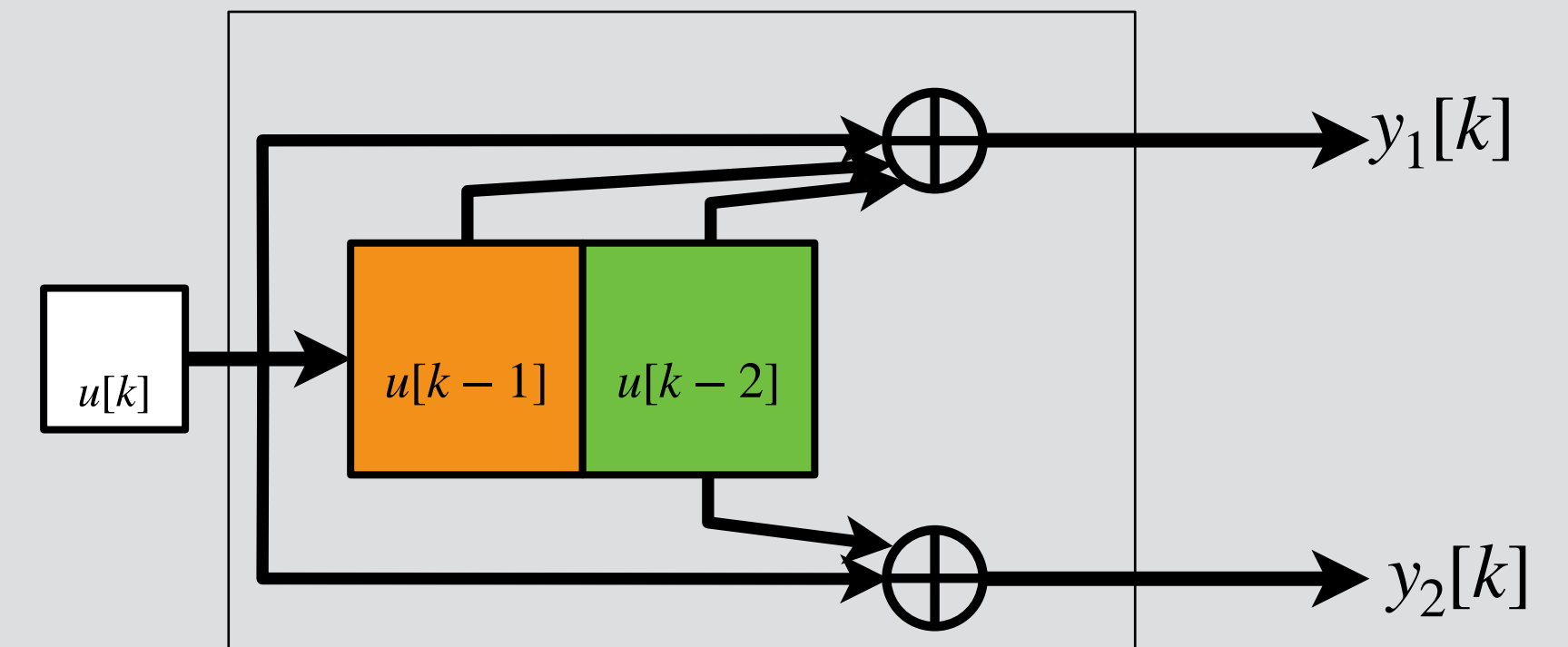
Infeasible quickly!

Trellis diagram

Data: $u = 10011$

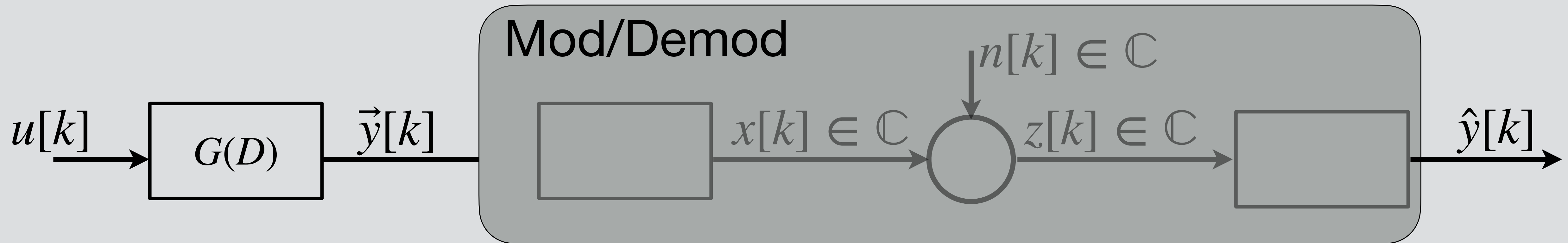
Sent: $y = 1110111101$

Tx bits set a path. Recover path, recover bits



Towards Decoding

Consider an AWGN channel with the following encoder / decoder:

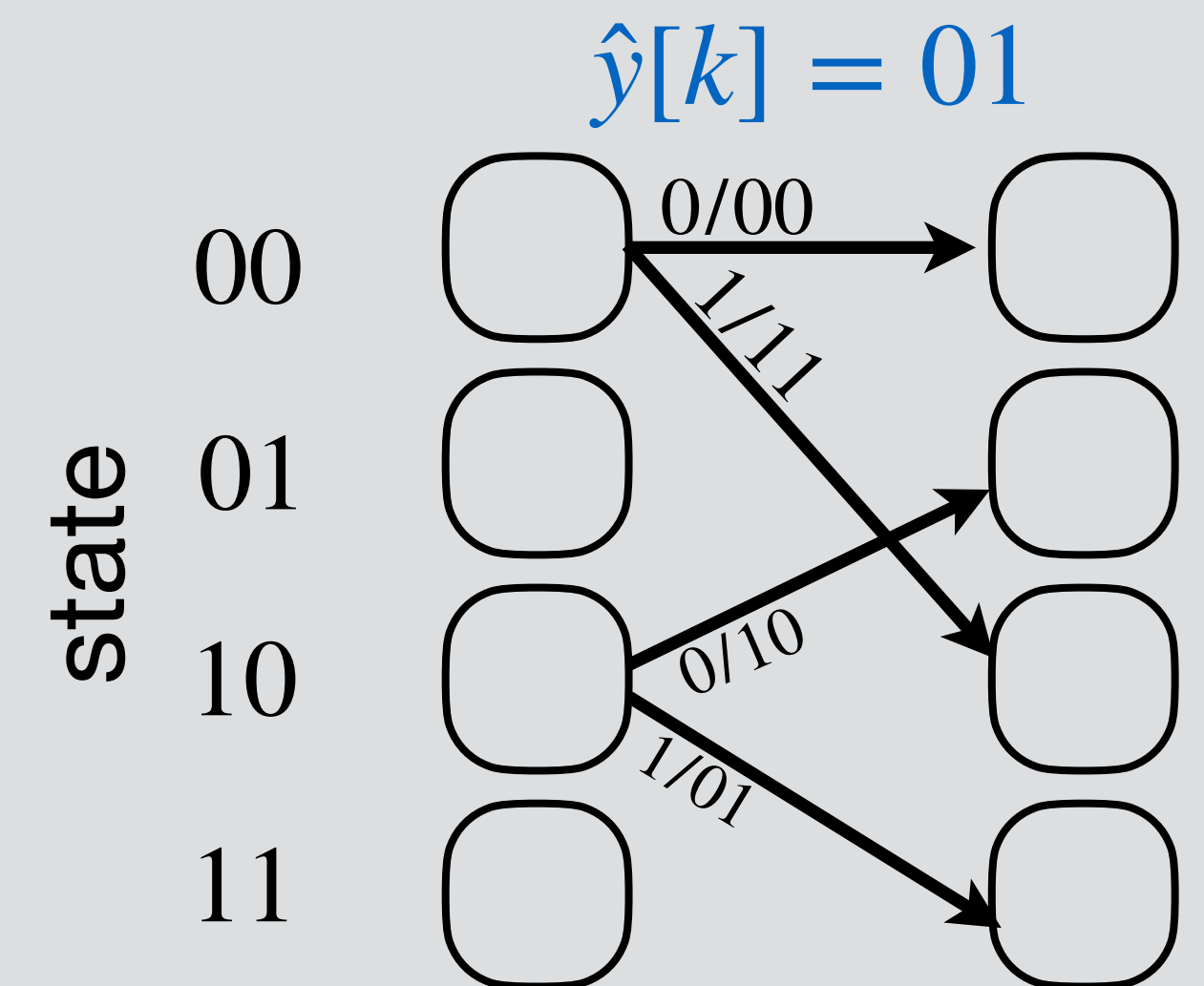


Define *branch metric* $\lambda_k(s[k] \rightarrow s[k+1])$

Cost of moving from one state to another:

$$\lambda_k(s[k] \rightarrow s[k+1]) = \sum_i \hat{y}_i[k] \oplus y_i[k]$$

Without noise, cost should be = 0!

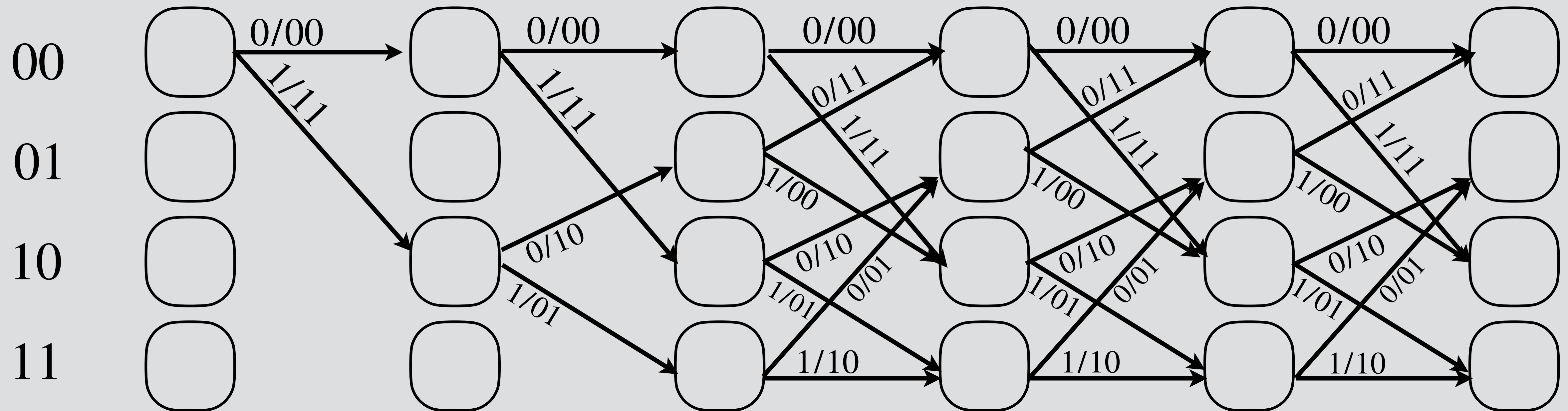
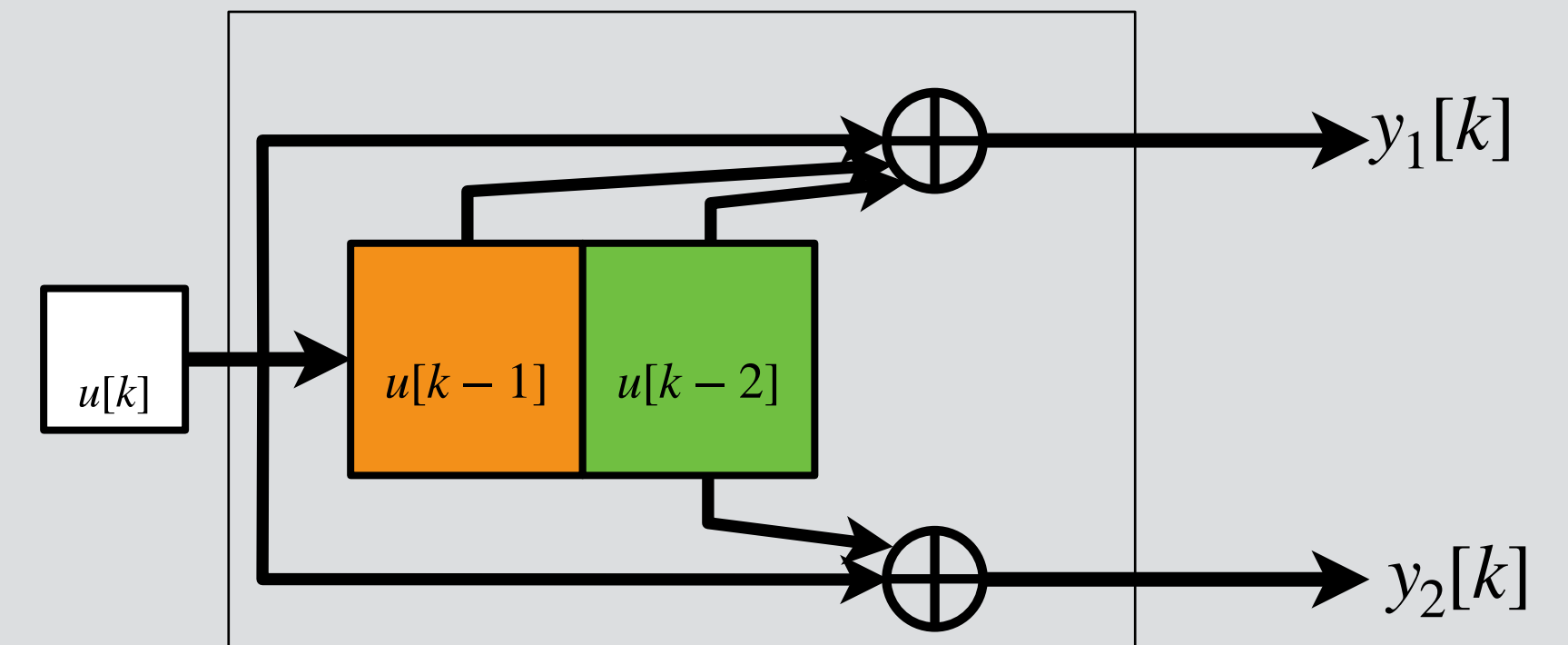


Trellis diagram and Viterbi Algorithm

Received: $z = 11101\mathbf{0}1101$

Exponential number of paths

Let's compute each branch metric (cost)

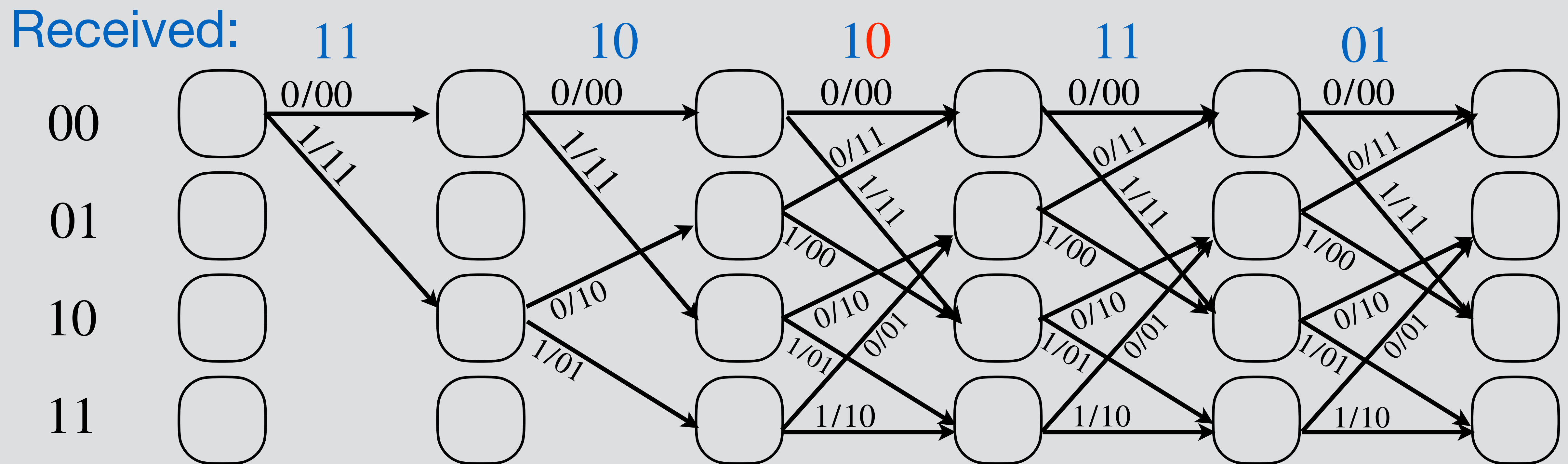
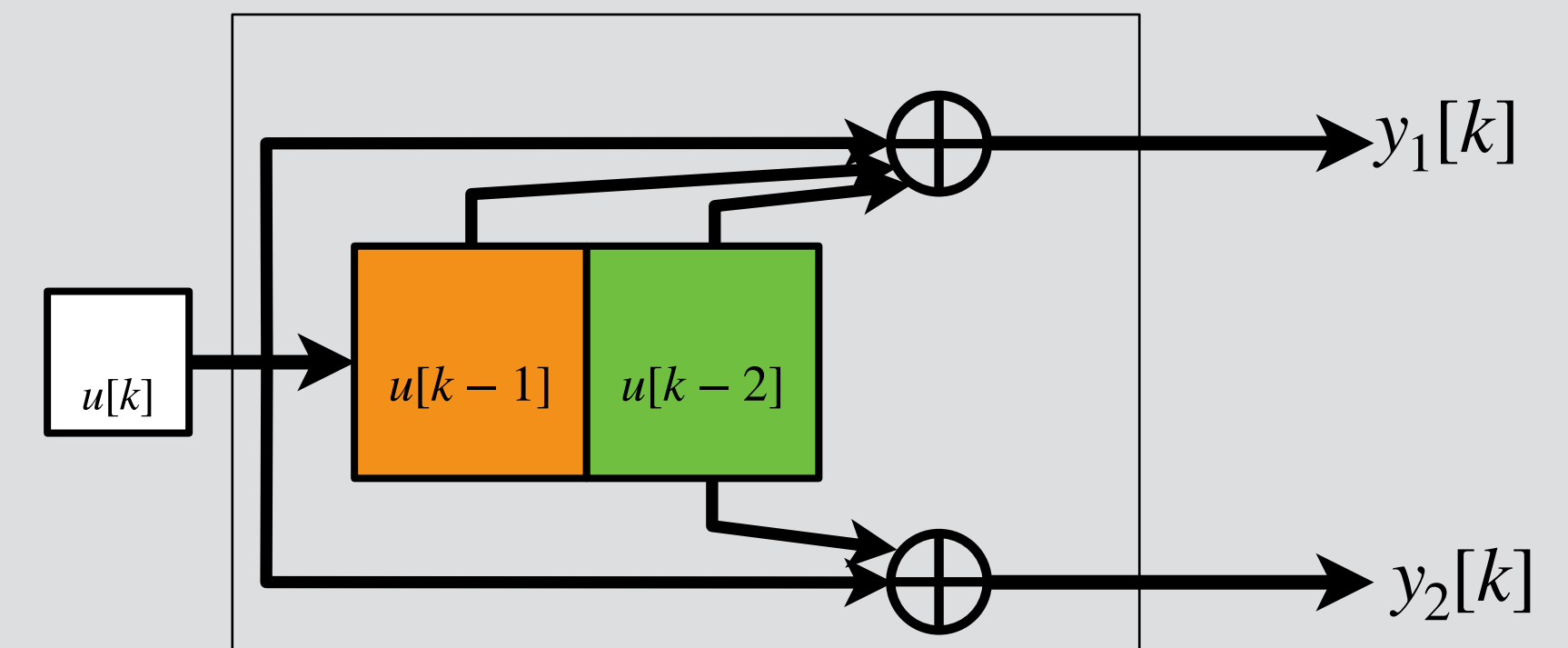


Trellis diagram and Viterbi Algorithm

Received: $z = 11101\mathbf{0}1101$

Exponential number of paths

Let's compute each branch metric (cost)

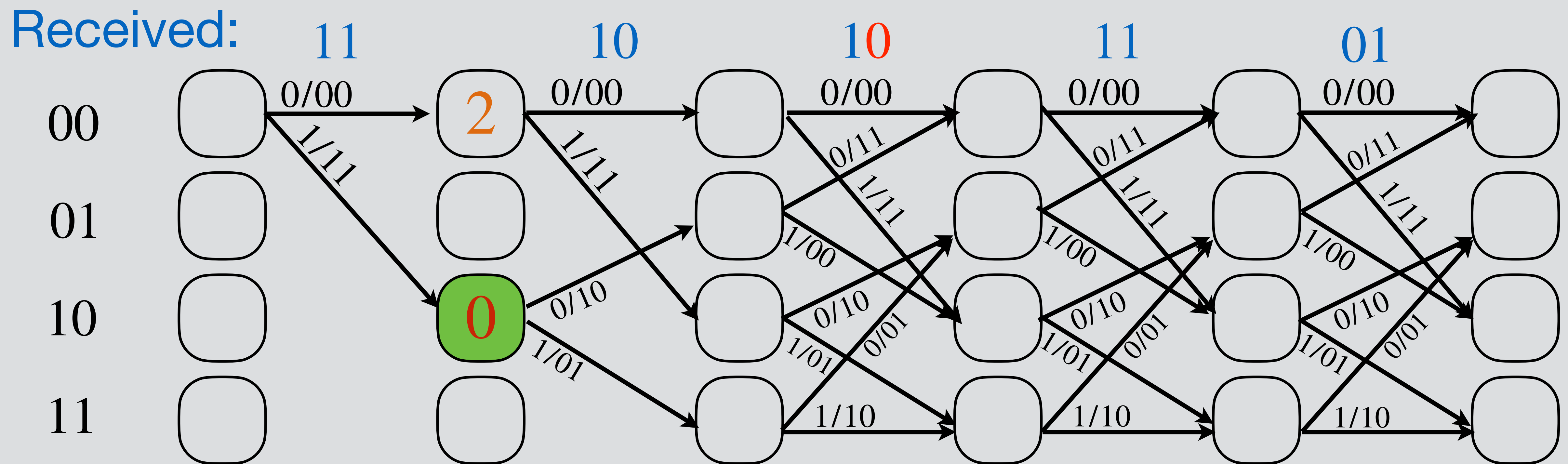
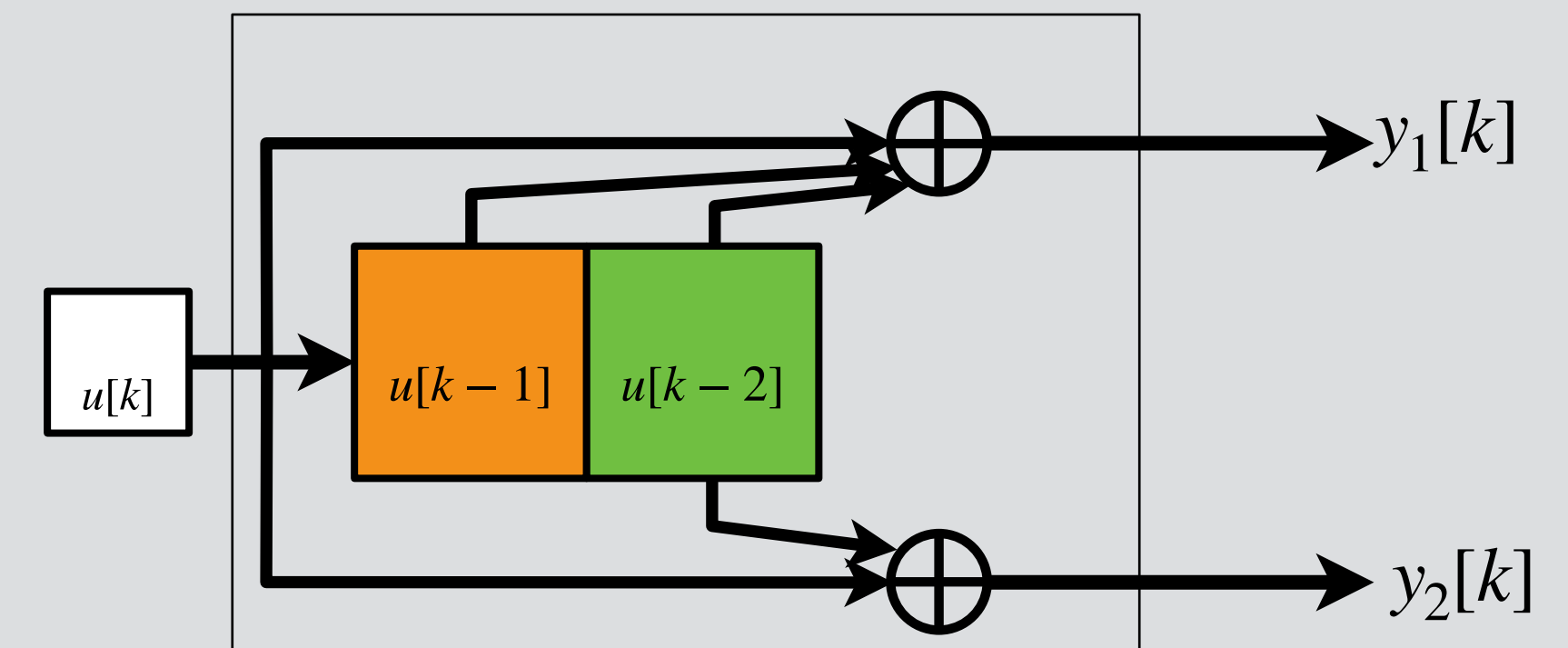


Trellis diagram and Viterbi Algorithm

Received: $z = 11101\mathbf{0}1101$

Exponential number of paths

Let's compute each branch metric (cost)

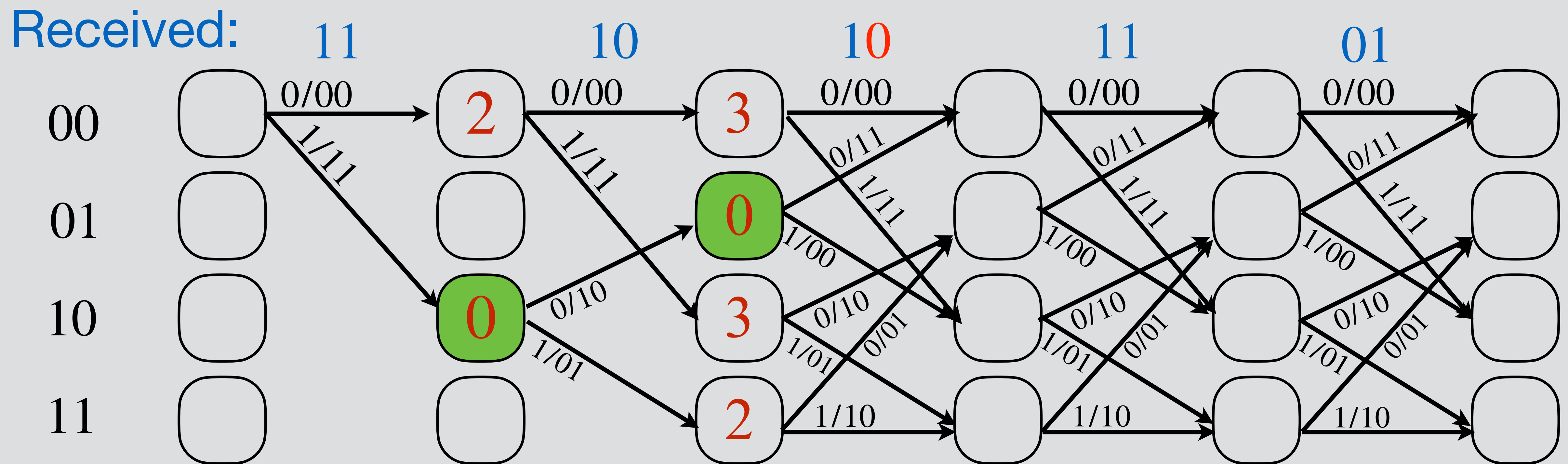
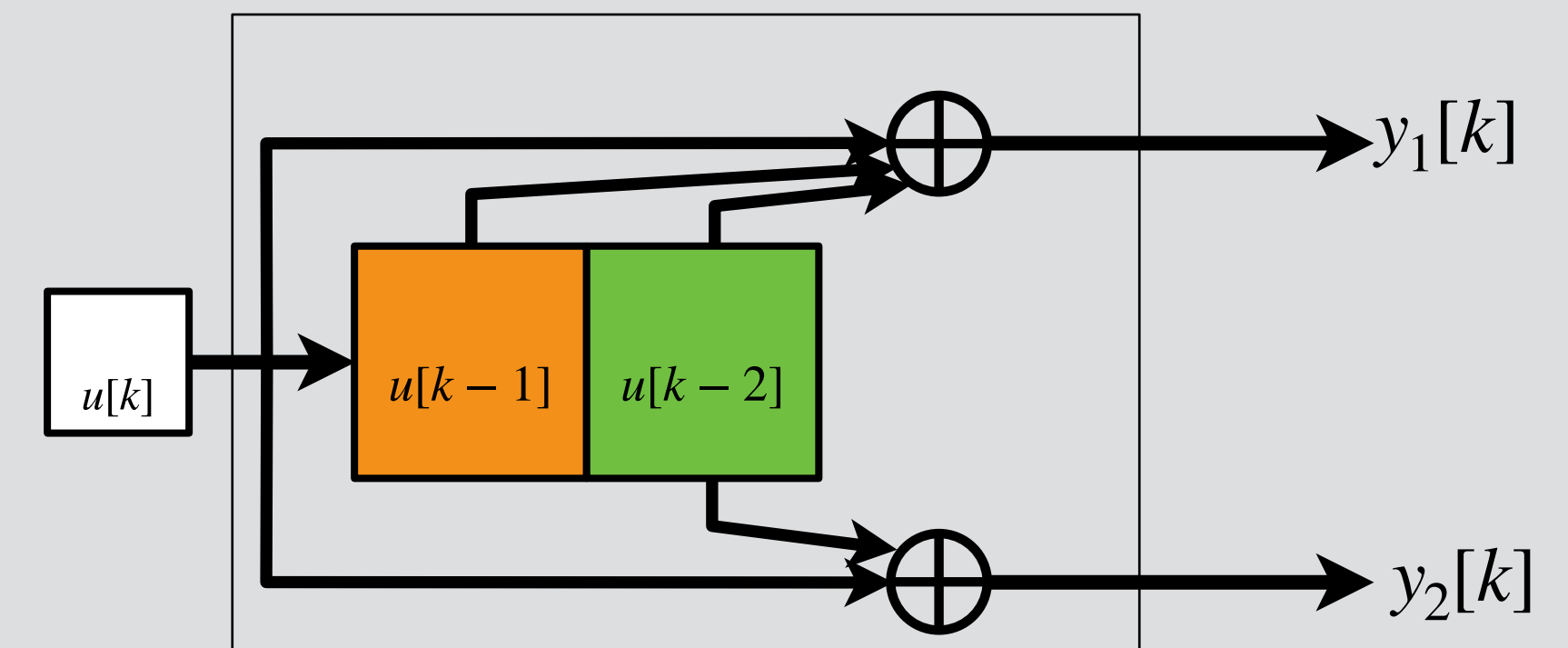


Trellis diagram and Viterbi Algorithm

Received: $z = 11101\mathbf{0}1101$

Exponential number of paths

Let's compute each branch metric (cost)

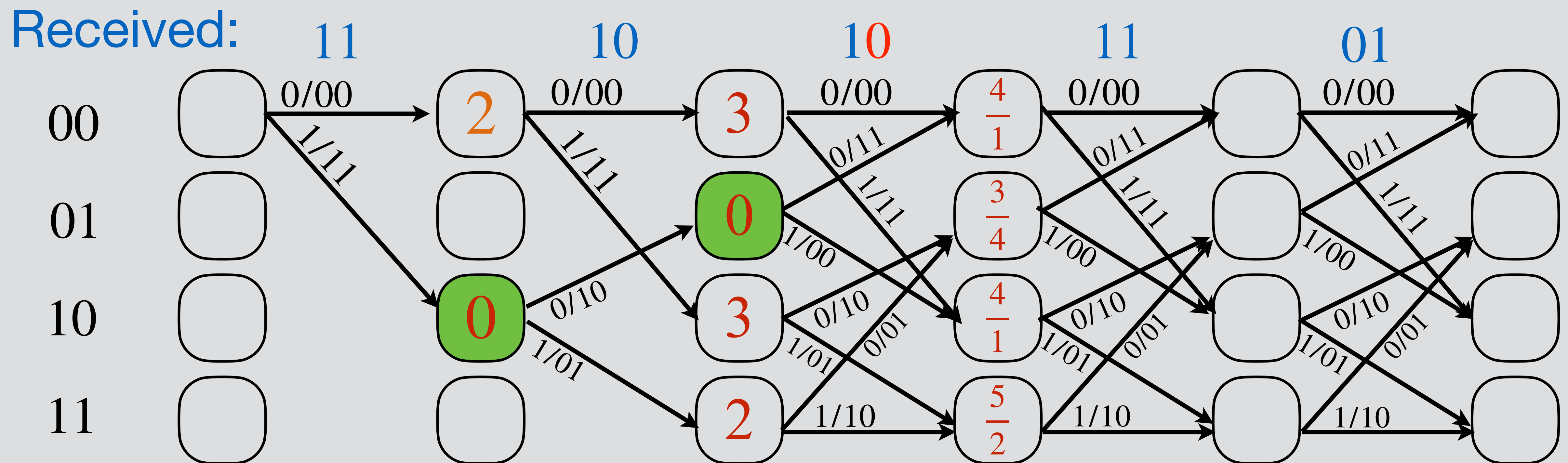
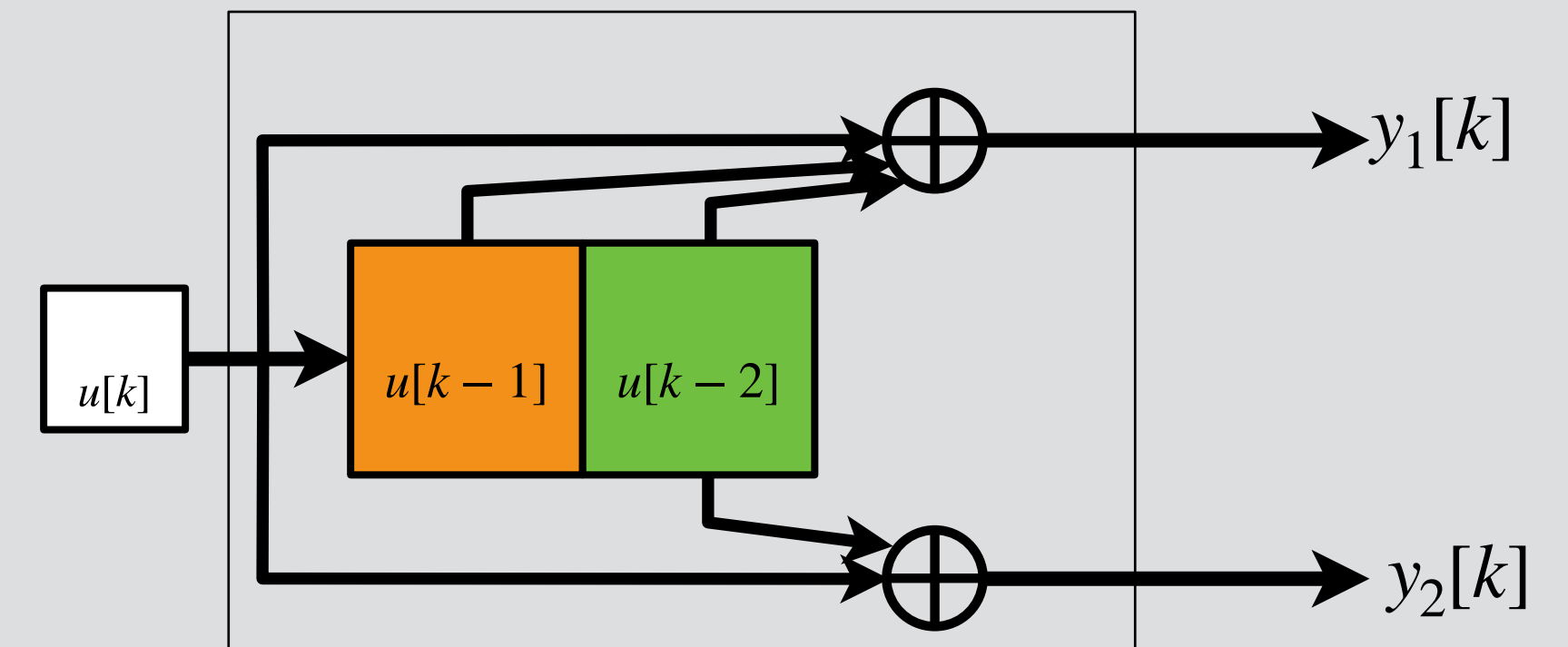


Trellis diagram and Viterbi Algorithm

Received: $z = 1110101101$

Exponential number of paths

Let's compute each branch metric (cost)



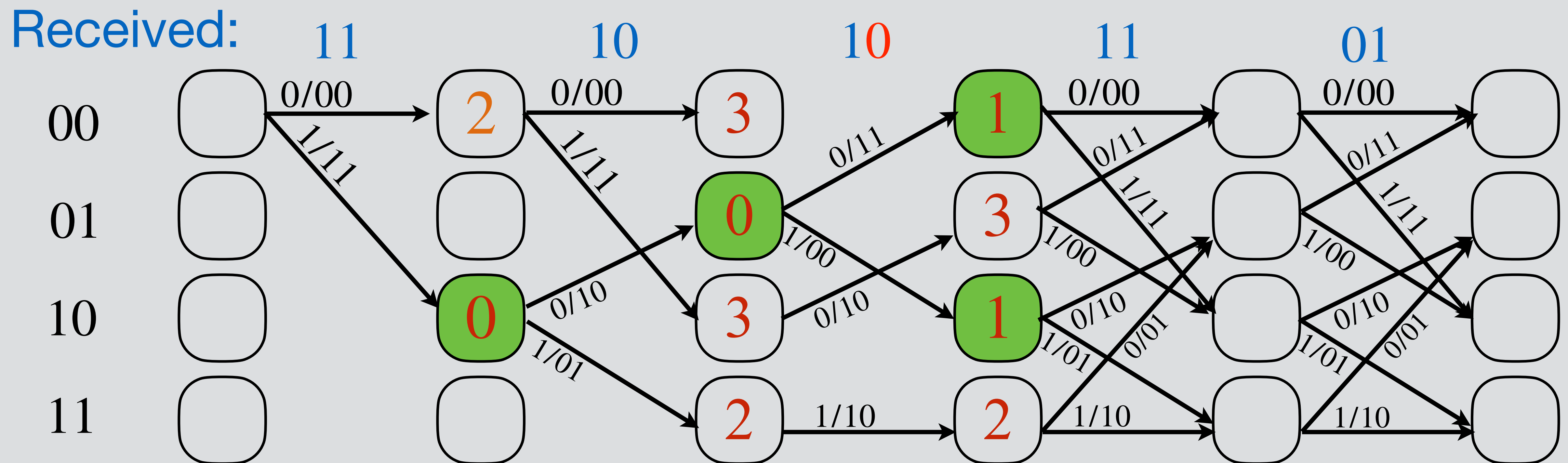
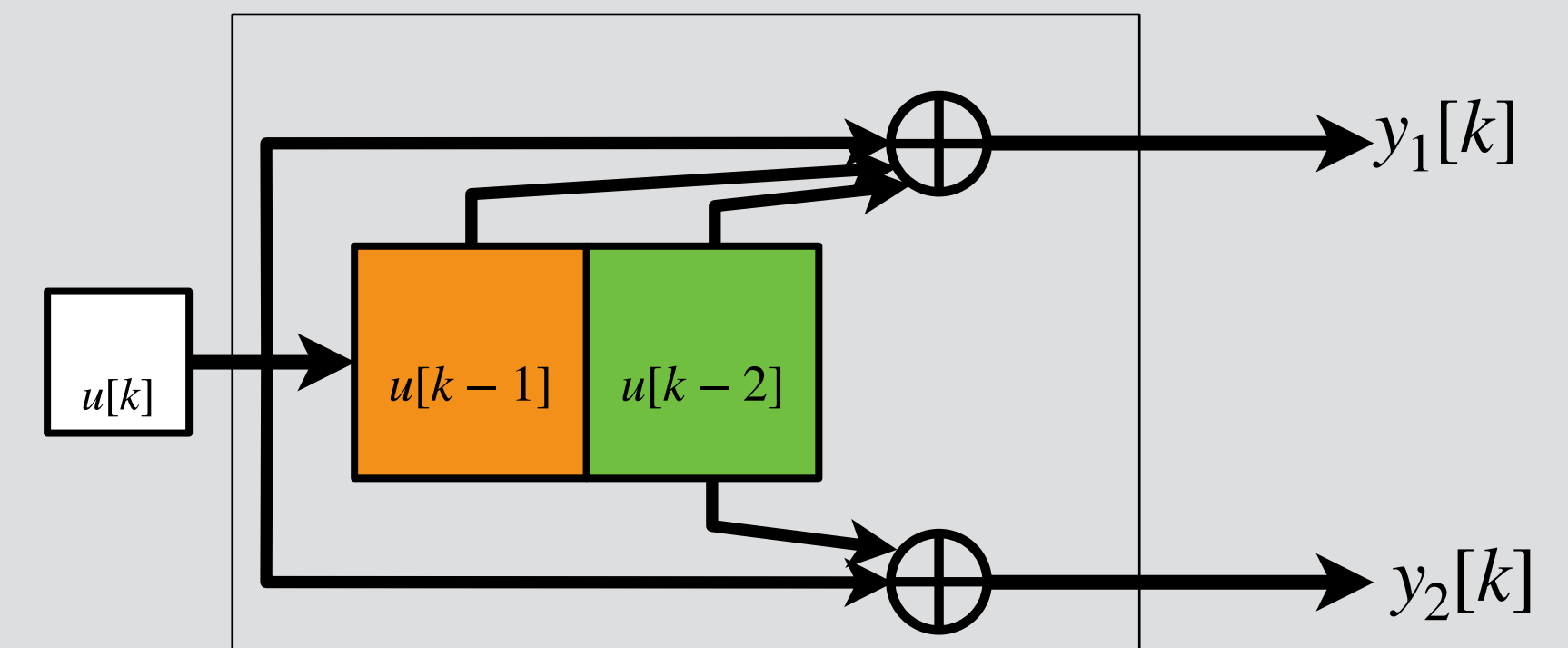
Don't need to keep track of losing paths!

Trellis diagram and Viterbi Algorithm

Received: $z = 11101\mathbf{0}1101$

Exponential number of paths

Let's compute each branch metric (cost)



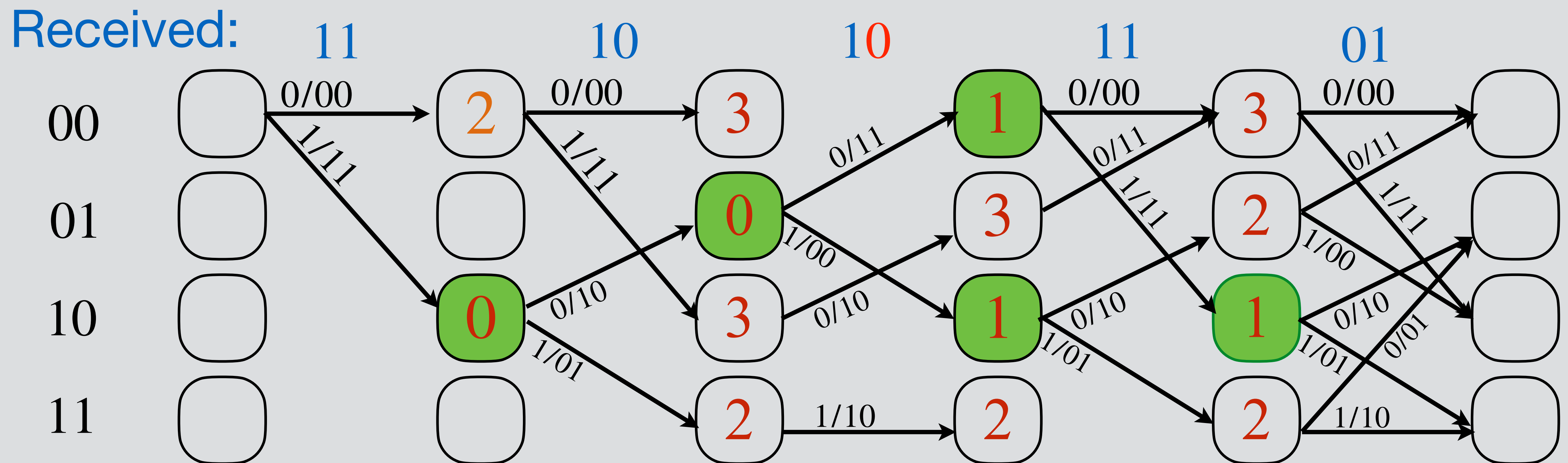
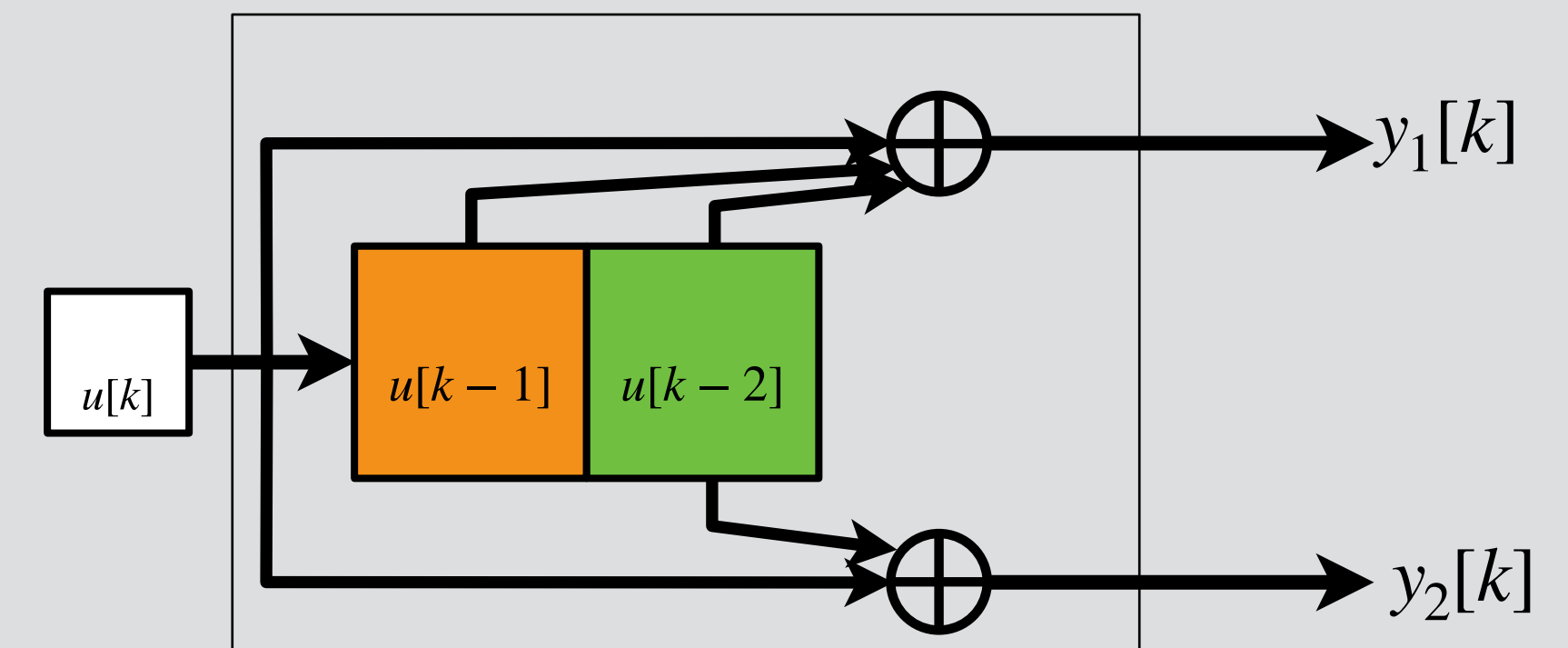
Don't need to keep track of losing paths!

Trellis diagram and Viterbi Algorithm

Received: $z = 11101\mathbf{0}1101$

Exponential number of paths

Let's compute each branch metric (cost)



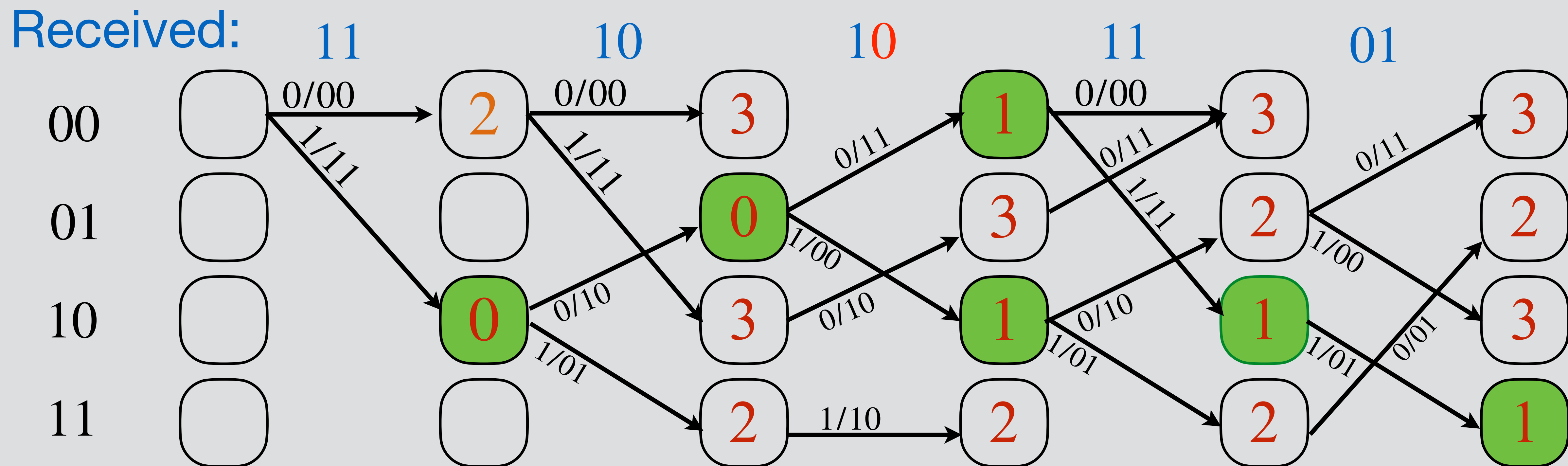
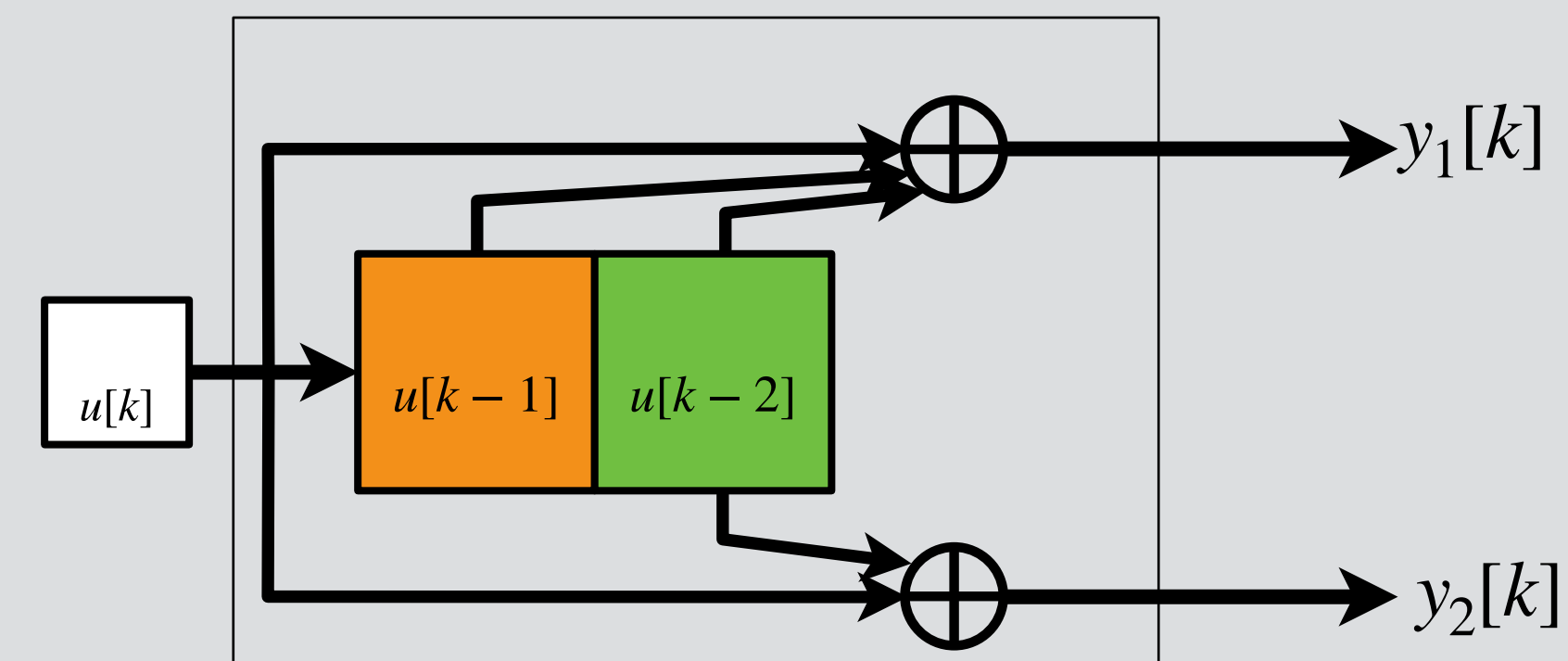
Don't need to keep track of losing paths!

Trellis diagram and Viterbi Algorithm

Received: $z = 11101\mathbf{0}1101$

Exponential number of paths

Let's compute each branch metric (cost)



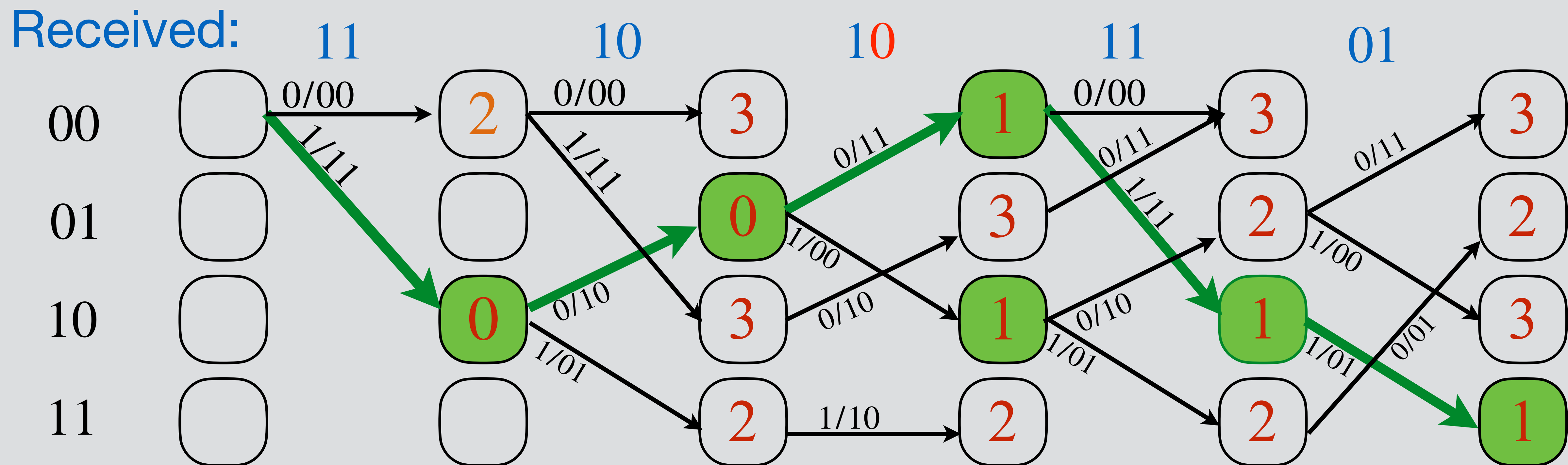
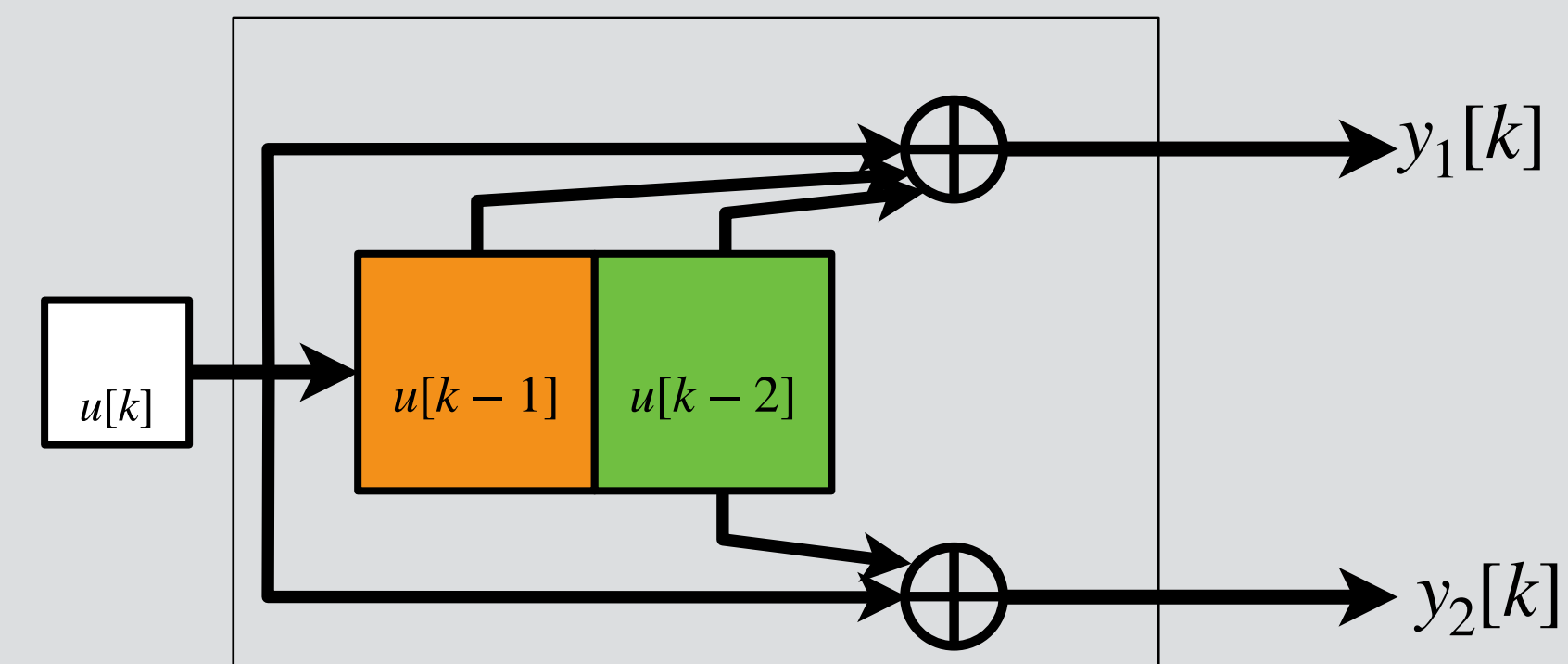
Don't need to keep track of losing paths!

Trellis diagram and Viterbi Algorithm

Received: $z = 11101\mathbf{0}1101$

Exponential number of paths

Let's compute each branch metric (cost)



Backtrack to find optimal path!

Trellis diagram and Viterbi Algorithm

Received: $z = 1110101101$

Corrected: 1 1 1 0 1 1 1 0 1

Decoded: $\hat{u} = 10011$

